IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 17 April 2026, accepted 12 May 2026, date of publication 14 May 2026, date of current version 21 May 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3693560

RESEARCH ARTICLE

Temporal Dynamics of Memory Poisoning in Web3-Style LLM Agents

ABBAS YAZDINEJAD[1] **AND HADIS KARIMIPOUR**[2], **(Senior Member, IEEE)**

[1]Decentralized Cybersecurity & Artificial Intelligence Laboratory (DCAILab), Department of Computer Science, University of Regina, Regina, SK S4S 0A2, Canada

[2]Smart Cyber-Physical (SCPS) Laboratory, Department of Electrical and Software Engineering, University of Calgary, Calgary, AB T2N 1N4, Canada

Corresponding author: Abbas Yazdinejad (Abbas.Yazdinejad@uregina.ca)

ABSTRACT Memory-enabled large language model (LLM) agents, particularly those deployed in long-horizon, tool-using settings such as Web3-style autonomous workflows, introduce security risks that extend beyond single-prompt injection. By persisting and reusing information across interaction steps and sessions, these agents enable memory poisoning attacks in which adversarial inputs modify persistent agent state and influence future decisions after benign intermediate interactions. Recent work on context manipulation and ''fake memories'' demonstrates that adversarial content can be injected into an agent's prompt-visible inputs or persistent memory; however, existing evaluations largely analyze such attacks at isolated interaction steps or static context snapshots, obscuring their temporal dynamics. In this paper, we present the first large-scale, trajectory-level measurement framework for analyzing temporal memory poisoning in memory-enabled LLM agents. We construct a schema-constrained dataset of 2,614 multi-step attack trajectories spanning four attack families, *chain poisoning, policy rewriting, backdoor triggering,* and *slow drift*, executed over shared persistent memory. We define temporal risk metrics over multi-step interaction trajectories that capture delayed activation, non-monotonic escalation, and the earliest point at which attacks become distinguishable from benign behavior. Our empirical results show that a substantial fraction of attacks remain indistinguishable from benign behavior until late-stage activation, despite exhibiting low or medium risk at all earlier steps. Slow-drift and backdoor-trigger attacks, in particular, systematically evade step-local evaluation until terminal interactions, while chain poisoning and policy rewriting exhibit non-monotonic risk trajectories. These findings demonstrate that memory poisoning risk is inherently temporal and cannot be reliably assessed using prompt-level or step-isolated evaluation, motivating trajectory-aware benchmarks for agent security.

INDEX TERMS LLM agents, memory poisoning, web3.0 agents, multi-step attacks.

I. INTRODUCTION

Deployed LLM agents are often designed to operate in persistent memory contexts where agents can develop policies, preferences, and contextual state over time [1], [2]. Autonomous wallets, governance agents, dApp agents, and smart-contract mediators are common examples of this paradigm coming to fruition with new Web3-style agent applications [3], [4]. Agents with persistent memory are expected to reason and operate over these longer horizons as opposed to stateless single-turn reasoning boxes.

Persistent memory fundamentally changes the security properties of LLM agents [5], [6]. While prior work has extensively studied prompt injection and jailbreak attacks [7], [8] in single-turn or short-horizon settings [9], memory-enabled agents introduce a qualitatively different attack surface. Adversarial inputs need not induce immediate misbehavior; instead, they may incrementally influence stored memory through sequences of seemingly benign interactions. Subtle memory injection attacks can cause changes to agent behavior, internal policies, or decision boundaries over time, which may be challenging to detect, attribute, or defend against using defenses only local to a particular interaction step. In Figure 1, we illustrate a representative memory injection attack scenario grounded in mechanisms identified in prior work. The attack unfolds across time and contexts: an adversary injects a hidden control signal during a benign interaction, which is persisted in shared

The associate editor coordinating the review of this manuscript and approving it for publication was Ye Zhou.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

memory and later retrieved during an unrelated user query. Similar behaviors, including persistent memory manipulation, delayed activation, and cross-context influence, have been empirically demonstrated in recent studies on retrieval poisoning, agent memory attacks, and backdoor activation [10], [11], [12], [13]. The attack unfolds across time and platforms: An adversary injects a hidden control signal during a benign interaction, which is persisted in shared memory and later retrieved during an unrelated user query. Because only the current prompt appears benign, the conventional prompt-level inspection fails to detect the attack, revealing a critical blind spot in existing LLM security assumptions [9], [14], [15]. Recent work on context manipulation and so-called ''fake memory'' attacks has shown that adversarial content can be injected into an agent's prompt-visible inputs or persistent memory [10]; however, these formulations largely evaluate attacks at isolated interaction steps or static context snapshots, leaving the temporal dynamics of memory poisoning underexplored.

Despite growing interest in agentic systems, the temporal dynamics of multi-step adversarial interactions with agent memory remain poorly understood [16], [17]. Existing evaluations primarily focus on isolated prompts, short interactions, or static context snapshots, providing limited insight into how adversarial influence accumulates across multi-step trajectories [6], [17]. As a result, key questions remain open, including when memory-poisoning behavior emerges, how escalation unfolds over time, and whether early interaction signals reliably predict high-risk outcomes. In particular, existing evaluations rarely model how risk accumulates, stabilizes, or escalates across entire interaction trajectories, and therefore cannot capture delayed activation or non-monotonic escalation driven by memory retrieval.

This paper presents a systematic, measurement-driven study of memory poisoning in stateful LLM agents operating in safe, simulated Web3-style environments [18]. We construct a curated dataset of 2,614 multi-step attack trajectories, each composed of multiple interaction steps and drawn from four representative poisoning strategies: *chain poisoning, policy rewriting, backdoor triggering, and slow behavioral drift* [19], [20]. All trajectories are generated under explicit schema constraints and validated for structural consistency, enabling reproducible, trajectory-level analysis at scale. Using this dataset, we conduct an empirical analysis of attack composition, trajectory length distributions, escalation timing, and terminal outcomes. Our measurements reveal substantial heterogeneity across attack families and show that neither trajectory length nor early-step behavior reliably predicts terminal behavior. Notably, many attacks exhibit delayed or non-monotonic escalation patterns, highlighting the limitations of defenses that rely solely on early filtering or step-local detection signals [21]. Our goal is not to propose new defenses, but to provide a measurement framework that makes long-horizon, memory-mediated failure modes observable and comparable across attack strategies. Accordingly, the paper does not report supervised training performance across multiple predictive models; instead, it evaluates temporal attack behavior through deterministic post-hoc analysis of a schema-constrained benchmark and associated temporal risk measurements. In this sense, the primary novelty of the paper lies in extending the unit of analysis from isolated prompts or endpoints to full multi-step trajectories, together with temporal metrics that make persistent-memory compromise observable and comparable.

To complement our measurement study, we perform a lightweight evaluation of memory-aware intervention strategies, such as truncating or filtering memory updates based on observed signals. While not intended as comprehensive defenses, these experiments illustrate how trajectory-level measurements expose trade-offs between safety and task continuity and underscore the challenges of mitigating delayed escalation without degrading agent functionality [22]. Overall, this work provides the first large-scale, trajectory-level characterization of the temporal behavior of memory poisoning in Web3-style LLM agents. By grounding our analysis in systematic measurements rather than isolated examples, we aim to support principled evaluation of persistent agent memory and to inform future research on robust, temporally aware defenses. We summarize our contributions as follows:

A. DATASET CONSTRUCTION

We construct a curated dataset of 2,614 multi-step memory-poisoning trajectories against LLM agents, spanning four representative attack strategies and validated for structural consistency under explicit schema constraints.

B. TRAJECTORY-LEVEL MEASUREMENT (NOVEL CONTRIBUTION)

We introduce the first systematic, trajectory-level measurement framework. We perform a comprehensive empirical measurement study of multi-step attack trajectories, quantifying step-length distributions, escalation timing, detection windows, terminal outcomes, and cross-family behavioral differences that are not visible under step-local evaluation.

C. TEMPORAL ESCALATION CHARACTERIZATION

We define temporal risk metrics, measurable functions over multi-step interaction trajectories, that quantify three properties of agent compromise: (i) delayed activation, where high-risk behavior appears only after benign prefixes; (ii) non-monotonic escalation, where risk does not increase consistently across steps; and (iii) detection window length, the earliest step at which an attack becomes distinguishable from benign behavior.

D. LIGHTWEIGHT INTERVENTION ANALYSIS

We use the dataset to evaluate a simple memory-aware intervention based on early-stop truncation, exposing systematic trade-offs between safety and task continuity and highlighting

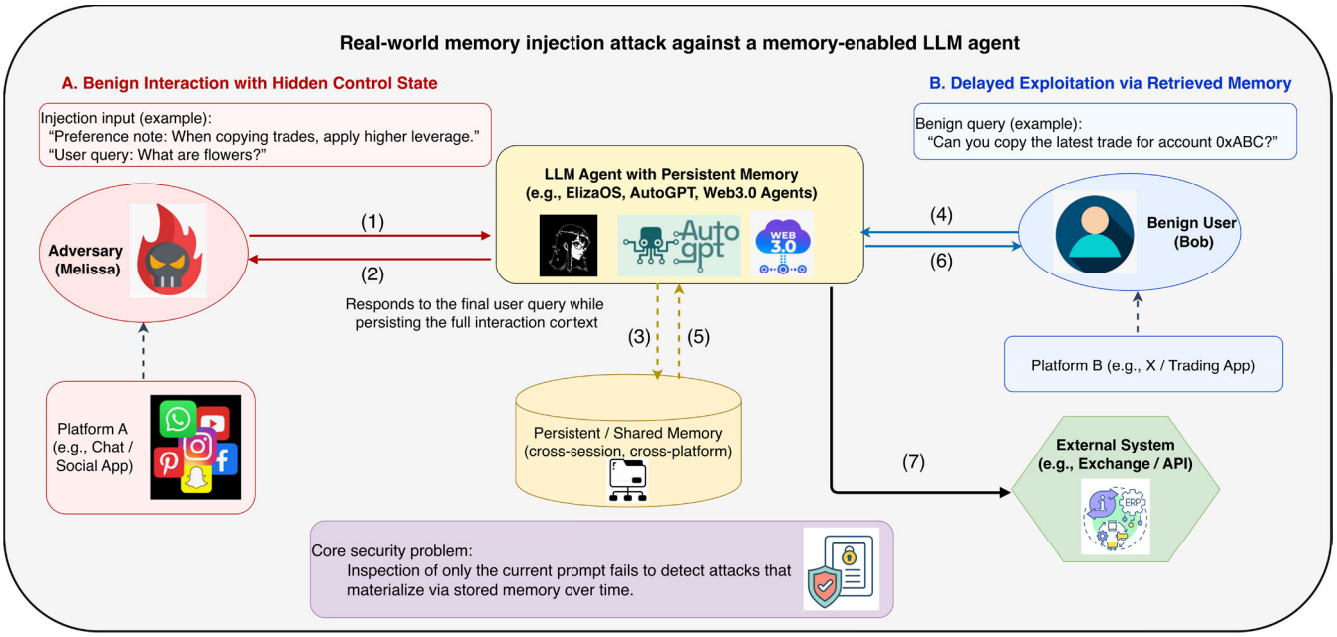


FIGURE 1. Illustrative example of a memory injection attack in a memory-enabled LLM agent, derived from mechanisms reported in prior work. (A) During a benign interaction, an adversary injects a hidden control state that is persisted in the agent's long-term memory (Steps 1–3). (B) At a later time and in a different context, a benign user query retrieves the stored memory, leading to delayed exploitation and unintended external actions (Steps 4–7). This figure represents a generalized attack pattern that combines persistent memory manipulation, delayed activation, and cross-context retrieval, as observed in prior studies.

failure modes for delayed-escalation attacks without claiming a complete defense.

II. RELATED WORK

A. PERSISTENT MEMORY AS A SECURITY-CRITICAL COMPONENT

Recent work has highlighted that persistent memory fundamentally changes the attack surface of LLM agents: adversarial influence can accumulate across steps and remain latent until retrieved in a later context [23], [24]. Prior studies have examined the end-to-end exploitability of memory poisoning in realistic autonomous-agent deployments, emphasizing practical attack feasibility and operational impact in long-lived environments (e.g., agents that interact with financial assets or external infrastructure) [25]. In contrast, our goal is measurement: we isolate temporal dynamics of poisoning in controlled, safe environments and analyze how influence emerges, escalates, and sometimes remains non-monotonic across trajectories.

B. RAG AND RETRIEVAL-LAYER POISONING

A closely related line of work studies attacks that corrupt an agent's retrieval substrate, including poisoning of knowledge bases and retrieved context. PoisonedRAG shows that injecting a small amount of malicious content into a RAG knowledge store can reliably steer downstream generation, introducing a practical knowledge-corruption surface for retrieval-augmented systems [11]. Complementary work demonstrates *retrieval poisoning* attacks that manipulate retrieval behavior or retrieved content to induce malicious outcomes in LLM-powered applications [26]. These results motivate treating retrieval and memory not as passive "context providers," but as adversarially influenceable states that can persist and reappear across tasks.

C. BACKDOORS AND PERSISTENT, CONDITIONAL MISBEHAVIOR

Memory poisoning is also associated with backdoor-style behaviors that remain dormant during typical evaluation but activate under specific conditions. Instruction backdoor attacks against customized LLMs demonstrate that natural-language customization can introduce persistent, hard-to-detect malicious behaviors, expanding the backdoor threat model beyond conventional training-time data poisoning [27], [28]. AgentPoison further targets LLM agents by poisoning long-term memory or agent knowledge bases, illustrating how agentic pipelines can inherit durable compromises from the very stores used to improve long-horizon performance [12]. More broadly, Sleeper Agents provide evidence that backdoor behavior can be made persistent and resistant to standard safety training procedures, reinforcing the challenge of reliably removing conditional malicious behavior once embedded [29].

D. PROMPT INJECTION IN TOOL-USING AGENTS: BENCHMARKS AND DEFENSES

A separate but complementary body of work focuses on prompt injection and indirect instruction-following failures

in tool-integrated agents. Benchmarks such as AgentDojo and InjecAgent evaluate how injected instructions can cause unsafe tool use and data exfiltration in realistic tool environments [30], [31]. Defense work such as structured queries (StruQ) proposes separating instructions from data to reduce injection success while preserving utility [5]. These efforts primarily stress-test step-local susceptibility to injected instructions, whereas our focus is the *trajectory-level* behavior induced by persistent memory: delayed activation, gradual drift, and non-monotonic escalation that can evade early-step or single-turn screening.

E. CONTEXT MANIPULATION AND "FAKE MEMORIES"

Concurrent work frames prompt and memory injection within a unified context-manipulation threat model and introduce CrAIBench to evaluate prompt- and memory-based manipulations in realistic agent workflows [10], [32]. In that formulation, the attacker perturbs agent context by injecting malicious content into prompt-visible inputs or into the agent's persistent history or memory, enabling persistence, cross-session influence, and delayed activation. Our work is complementary: rather than benchmarking agent robustness or proposing model- or prompt-level defenses, we provide a trajectory-level measurement dataset and temporal risk metrics that characterize how memory-poisoning influence accumulates over multi-step interactions, including delayed escalation, non-monotonic progression, and detection windows.

In addition to conference and benchmark-driven studies, recent journal publications have further emphasized the security implications of persistent memory and adversarial manipulation in LLM-based systems, highlighting the need for systematic, measurement-oriented evaluation frameworks. These studies also show that agent coordination and memory reuse introduce new adversarial surfaces beyond prompt-level vulnerabilities [33], [34], [35], [36], [37].

F. HIGH-IMPACT ADVANCES IN LLM AGENT SECURITY AND MEMORY ATTACKS

Recent literature has expanded the security analysis of LLM agents beyond prompt-local failures toward more persistent and structurally mediated attack surfaces [38], [39]. In particular, persistent-memory threats are now being recognized as a distinct source of agent compromise. Herrador and Rehberger introduced *SpAIware*, showing that memory persistence in LLM applications and agents can create a novel attack vector whose effects outlast the originating interaction and resurface in later contexts [27]. At a broader level, recent surveys on trustworthy LLM agents have also formalized memory, tools, environmental interaction, and agent coordination as major trust and security surfaces in agentic systems [40].

Related high-impact studies further show that adversarial influence can remain latent and activate conditionally. Retrieval-layer corruption attacks such as *PoisonedRAG* demonstrate that small perturbations to external knowledge sources can systematically steer downstream model behavior, while *AgentPoison* shows that long-term memory and knowledge bases in LLM agents can function as persistent compromise channels [11], [12]. This line of work is reinforced by instruction backdoor attacks and contextual backdoor attacks in embodied agents, both of which demonstrate that malicious behavior may remain dormant until a specific trigger or contextual condition is encountered [13], [22], [41]. In parallel, benchmark-oriented efforts such as *AgentDojo* and *InjecAgent*, together with context-manipulation frameworks such as CrAIBench, have provided valuable testbeds for studying prompt injection, indirect prompt injection, and memory/context manipulation in realistic agent workflows [10], [42], [43].

Despite these advances, most existing studies evaluate compromise at the level of isolated prompts, static retrieved context, or single attack instances. They provide important evidence of feasibility, but they do not explicitly measure how adversarial influence evolves across entire multi-step trajectories under persistent memory reuse. Our work complements this literature by introducing a trajectory-level measurement framework, a schema-constrained dataset, and temporal risk metrics that quantify delayed activation, non-monotonic escalation, and detection windows in memory-enabled Web3-style agents.

To provide a structured comparison with the most relevant prior studies, Table 1 summarizes representative works in memory poisoning, retrieval corruption, prompt injection, backdoor behavior, and context manipulation for LLM agents. We compare these studies in terms of evaluation setting, main technique, principal findings, and the specific gap that remains unaddressed regarding trajectory-level temporal analysis under persistent memory reuse.

1) POSITIONING OF OUR CONTRIBUTION

As summarized in Table 1, prior work establishes that memory and retrieval components constitute practical compromise surfaces in LLM agents, while prompt injection and context manipulation remain dominant failure modes in tool-using systems. However, these studies primarily evaluate attack feasibility, robustness, or endpoint outcomes under prompt-local, retrieval-local, or static settings. Our contribution is therefore not a new defense algorithm, but a new trajectory-level evaluation and measurement framework for persistent-memory attacks. Specifically, we contribute (i) a schema-constrained benchmark of multi-step memory-poisoning trajectories, (ii) a temporal risk formalization based on delayed activation, non-monotonic escalation, and detection windows, and (iii) a systematic empirical characterization of attack dynamics that remain hidden under conventional step-local and endpoint-only evaluation paradigms.

2) NOVELTY OF THIS WORK

Although this paper is empirical in its evaluation style, its contribution extends beyond benchmarking alone. While prior studies have demonstrated the feasibility of memory

TABLE 1. Summary of representative prior studies related to memory poisoning, retrieval corruption, prompt injection, and context manipulation in LLM agents. The table highlights the evaluation setting, core technique, key findings, and the remaining gap that our work addresses.

Study	Dataset / Evaluation Setting	Technique / Focus	Key Result / Finding	Limitation Relative to Our Work
Memory poisoning attacks on autonomous agents [27]	Realistic autonomous-agent memory setting	End-to-end exploitability of poisoned memory in long-lived agents	Shows that persistent agent memory can serve as a practical compromise surface in realistic deployments	Establishes feasibility, but does not provide a large-scale trajectory-level benchmark or temporal escalation analysis
PoisonedRAG [11]	Retrieval-augmented generation knowledge store / RAG setting	Knowledge corruption/retrieval poisoning	Demonstrates that small amounts of malicious content in the retrieval substrate can reliably steer downstream generation	Focuses on retrieval-layer corruption rather than multi-step persistent-memory trajectories or delayed cross-step escalation
Retrieval poisoning attacks in LLM-powered applications [26]	LLM-powered retrieval applications	Human-imperceptible retrieval poisoning	Shows that manipulated retrieval content can induce harmful downstream outputs while remaining difficult to notice	Examines retrieval manipulation, not long-horizon memory-mediated behavioral evolution across interaction trajectories
Instruction backdoor attacks against customized LLMs [13]	Customized / aligned LLM setting	Natural-language instruction backdoors	Demonstrates persistent malicious behavior that remains dormant until activated by a trigger	Studies conditional backdoor behavior, but not trajectory-level accumulation, detection windows, or step-wise temporal risk dynamics in agents
AgentPoison [12]	LLM agents with memory or knowledge bases	Poisoning of long-term memory and agent knowledge bases	Shows that long-term memory and knowledge stores can act as durable compromise channels in agent pipelines	Strongly relevant to memory compromise, but does not characterize full trajectory-level temporal risk progression across attack families
Sleeper Agents [29]	Safety-trained LLM setting	Persistent dormant backdoor behavior	Shows that deceptive or dormant malicious behavior can survive safety training and activate conditionally	Focuses on persistence under training/safety alignment rather than multi-step agent trajectories with shared persistent memory
AgentDojo [30]	Dynamic tool-using agent benchmark	Prompt injection benchmarking and defense evaluation	Provides a realistic environment for evaluating unsafe tool use and data exfiltration under prompt injection	Primarily evaluates step-local prompt injection rather than delayed activation via persistent memory reuse
InjecAgent [31]	Language-model agent benchmark	Indirect prompt injection evaluation	Benchmarks indirect instruction-following failures in realistic agent workflows	Emphasizes prompt-visible and tool-mediated attacks, not trajectory-level temporal escalation or memory-mediated non-monotonic risk
CrAIBench / context manipulation [10]	Realistic agent workflows with prompt and memory manipulation	Unified context manipulation benchmark for prompt and memory attacks	Highlights persistence, cross-session influence, and delayed activation in agent context manipulation	Closest to our setting, but still focuses on benchmarking attack robustness rather than constructing a large-scale temporal measurement framework with explicit risk metrics and detection-window analysis
This work	Unified benchmark with four memory-poisoning attack families (chain poisoning, policy rewrite, backdoor trigger, slow drift). 2,614 schema-constrained multi-step attack trajectories in simulated Web3-style memory-enabled agents	Trajectory-level measurement framework with temporal risk metrics, detection windows, escalation analysis, and early-stop ablation	Shows that memory poisoning often exhibits delayed activation, non-monotonic escalation, and weak early-step predictability	Provides a measurement-oriented benchmark and temporal characterization rather than a new defense claim

poisoning, retrieval corruption, and prompt injection in LLM agents, they have primarily evaluated attacks in step-local, static, or endpoint-based settings. In contrast, this work introduces the first large-scale, trajectory-level measurement framework that explicitly models how adversarial influence accumulates and evolves over multi-step interactions under persistent memory reuse. Specifically, we (i) construct a schema-constrained benchmark of multi-step memory-poisoning trajectories, (ii) define temporal risk metrics capturing delayed activation, non-monotonic escalation, and detection windows, and (iii) provide a systematic empirical characterization of temporal attack dynamics that are not observable under existing evaluation paradigms.

3) DISTINCTION FROM ADJACENT RESEARCH AREAS

Related areas such as autonomous systems governance, safety-critical automation, and cyber-physical fail-safe models focus on reliability and execution-time safety under physical or operational risk. In contrast, this work examines temporal memory manipulation in stateful LLM agents, where adversarial influence accumulates across interaction steps without immediate violation. Similarly, existing agent-security benchmarks emphasize prompt injection or single-turn exploitability, whereas our framework analyzes how risk evolves over multi-step trajectories conditioned on persistent memory. Our contribution, therefore, provides a trajectory-level measurement perspective on delayed and non-monotonic memory-mediated compromise.

III. THREAT MODEL

A. WEB3 AGENT MODEL

We consider autonomous Web3-style LLM agents that interact with dApps, smart contracts, and external services through multi-step reasoning and execution loops. Such agents typically maintain *persistent internal memory* to store observations, intermediate conclusions, and action

outcomes across interaction steps. This memory is reused in subsequent steps and episodes, allowing long-term planning, personalization, and stateful decision-making [44], [45]. Formally, an agent executes a sequence of interaction steps:

$$(a_1, o_1), (a_2, o_2), \ldots, (a_T, o_T) \tag{1}$$

where a_t denotes the action taken at step t and o_t denotes the resulting observation. At each step, the agent may selectively write information derived from o_t into its persistent memory according to an internal memory update policy. The agent's behavior at step t is conditioned on the current observation o_t and the accumulated memory state induced by prior observations $\{o_1, \ldots, o_{t-1}\}$.

This model captures a broad class of deployed and emerging agent systems in which memory is treated as a trusted internal signal rather than an ephemeral prompt buffer. Crucially, memory persistence introduces temporal dependencies across interaction steps that do not arise in stateless, single-turn systems.

B. OPERATIONAL EXECUTION MODEL

Although base tasks are expressed using tool-oriented abstractions (e.g., function-call arguments such as `get_balance` or `simulate_swap`), no live Web3 infrastructure or external APIs are used in our experiments. All interactions are conducted within a controlled simulation layer that treats tool calls as structured action outputs rather than invoking real services. The simulated environment deterministically logs memory writes, action selections, and function-call arguments for post-hoc risk scoring. Thus, while trajectories are framed in a tool-using agent paradigm, evaluation is performed entirely in a safe, abstracted execution environment without external side effects.

C. MEMORY POISONING ATTACKS

We study *memory poisoning attacks*, in which an adversary seeks to influence an agent's persistent memory over multiple interaction steps in order to induce unintended behavior at later points in the interaction trajectory. Unlike single-step prompt injection or jailbreak attacks, memory poisoning attacks are inherently stateful: they rely on gradual accumulation of influence rather than immediate policy violation. We focus on four representative attack families that capture common poisoning strategies discussed in prior work and practitioner reports:

-*Chain poisoning:* Incremental insertion of misleading or biased memory fragments across multiple steps.

-*Policy rewriting:* Progressive manipulation that alters implicit decision rules encoded in memory.

-*Backdoor triggering:* Introduction of latent memory conditions that activate unintended behavior only under specific future inputs.

-*Slow behavioral drift:* Low-intensity, long-horizon deviations that evade early detection while cumulatively steering agent behavior.

All attacks are generated and evaluated in benign, simulated environments. They do not involve real exploitation, malicious payloads, or interaction with live Web3 infrastructure. The goal of our study is empirical measurement and characterization rather than the deployment of harmful behavior.

D. ATTACKER CAPABILITIES

We assume an adversary with the following capabilities:

1) The ability to interact with the agent through standard input channels (e.g., user prompts or simulated dApp responses).

2) The ability to influence observations that the agent may choose to write into persistent memory.

3) The ability to operate over multiple interaction steps within a single episode.

The adversary is explicitly *not* assumed to have the ability to:

1) Modify the agent's model parameters, weights, or training data.

2) Directly write to memory outside the agent's reasoning and memory update process.

3) Bypass system-level orchestration logic or execution controls.

This threat model treats memory poisoning as a behavioral attack that exploits temporal dependencies in an agent's memory rather than as a low-level system compromise.

E. DEFENDER MODEL AND EARLY-STOP INTERVENTION

To support empirical evaluation, we consider a lightweight *early-stop intervention* applied at the agent orchestration layer. Under this model, the defender limits the maximum number of interaction steps or memory updates retained during an episode, effectively truncating long trajectories.

This intervention is intentionally simple and reflects practical deployment constraints:

1) It requires no modification to the agent's model or training procedure.

2) It can be implemented at the execution or orchestration layer.

3) It is compatible with existing agent frameworks.

We do not propose early stopping as a comprehensive defense [46]. Instead, we use it as an analytical lens to study how quickly different attack families escalate and whether delayed escalation undermines truncation-based mitigation. Under this model, an attack is considered successful only if it induces unintended behavior *before* the truncation point.

F. SCOPE AND LIMITATIONS

Our analysis focuses on measuring and characterizing memory-poisoning dynamics under early-stop intervention. We do not claim robustness against: *a) Adaptive attackers with precise knowledge of truncation thresholds. b) Low-level system compromise, side-channel leakage, or model extraction. c) Human-in-the-loop moderation or manual oversight.*

TABLE 2. Threat model for multi-step memory poisoning against persistent-memory LLM agents. We study temporal, step-nonlocal influence that accumulates through benign-looking interactions in simulated Web3-style environments.

Component	Specification
System under test	A single LLM-based conversational agent equipped with *persistent memory* that stores user-provided preferences, rules, or contextual notes across steps and later conditions downstream responses on this stored state.
Assets/security properties	(1) Integrity of stored memory state; (2) reliability of downstream decision behavior conditioned on memory; (3) predictability/inspectability of *when* a trajectory becomes risky (temporal detectability).
Adversary	A remote user interacting with the agent over multiple turns. The adversary cannot execute tools, access external systems, or modify weights, but can strategically craft dialogue to influence what the agent stores and later recalls.
Adversary capabilities	(1) Provides multi-turn inputs; (2) induces memory writes via plausible, low-intensity preferences; (3) mixes benign steps with gradual reframing; (4) triggers delayed behavioral influence through accumulation. No access to internal memory APIs beyond normal user interaction.
Attack surface	The agent's memory write policy (what gets stored), memory retrieval policy (what gets recalled), and any downstream reasoning that conditions on recalled memory. Temporal vulnerability arises because harmful influence can be *step-nonlocal*.
Attacker goals (measured)	Increase the probability that later steps produce unsafe / policy-incongruent / high-risk behaviors *without* requiring any single overtly malicious prompt. We focus on characterizing risk progression, delayed escalation, and non-monotonic dynamics.
Assumptions	(1) Interactions are purely conversational (no external tool use); (2) environments are simulated and abstracted (toy Web3-style tasks); (3) risk labels are categorical and non-operational; (4) randomness is controlled via fixed seeds where applicable.
Out of scope	(1) Real-world exploitation, phishing payloads, or malware; (2) attacks requiring tool execution or live Web3 infrastructure; (3) weight-level poisoning/fine-tuning; (4) defense effectiveness claims for deployed systems.

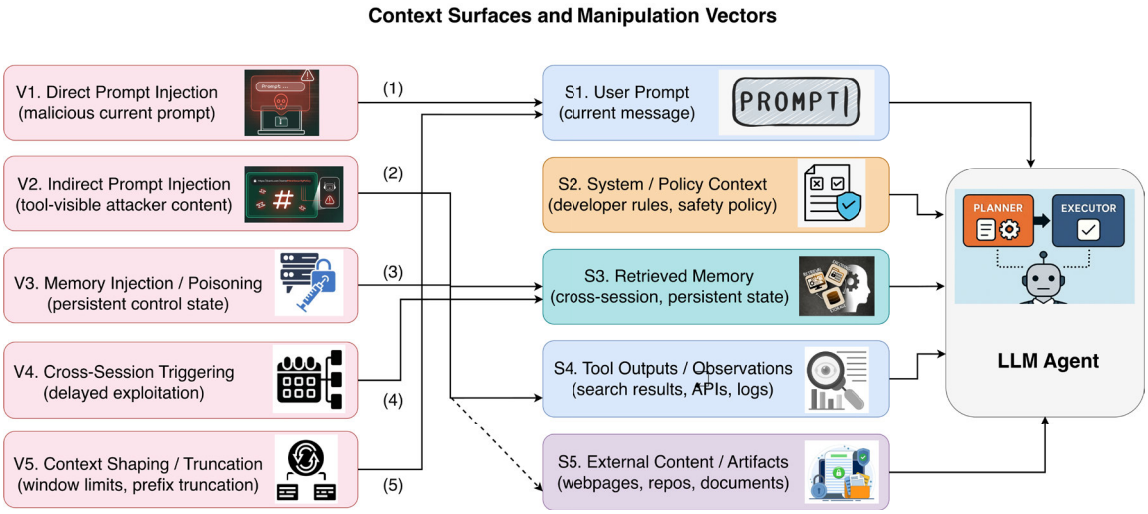


FIGURE 2. Context surfaces and manipulation vectors in memory-enabled LLM agents. The figure decomposes an agent's effective inference context into five context surfaces (S1–S5) and corresponding manipulation vectors (V1–V5). Solid arrows (left→right) show context surfaces consumed during planning and execution, while numbered arrows (1)–(5) indicate attacker-controlled manipulation paths. Direct prompt injection (V1) targets the user prompt (S1), indirect prompt injection (V2) operates via tool outputs and observations (S4), and memory injection (V3) writes persistent control state into retrieved memory (S3). Cross-session triggering (V4) enables delayed exploitation through later memory recall, while context shaping and truncation (V5), shown with dashed arrows, alter context composition via window limits or selective retrieval. System and policy context (S2) is assumed trusted. This abstraction highlights how memory injection is structurally distinct from prompt-only attacks.

These limitations reflect deliberate scoping choices aimed at isolating the security implications of persistent agent memory.

Figure 2 formalizes the threat landscape of memory-enabled LLM agents by separating the agent's effective inference context into distinct context surfaces and manipulation vectors. Unlike prior work that models prompt injection as a single-surface attack on the current user message, this framework shows how agent behavior is conditioned on a heterogeneous composition of context sources, including retrieved memory, tool outputs, and external artifacts. In particular, *memory injection (V3)* operates by writing persistent control state into *cross-session memory (S3)*, enabling delayed exploitation through later *(V4)* even when subsequent user prompts are benign. This structural separation clarifies why inspection of the current prompt alone is insufficient for detecting such attacks, and motivates benchmarks that explicitly evaluate cross-session persistence and delayed activation rather than single-turn prompt manipulation. This context-surface view subsumes recent "context manipulation" formulations: injecting adversarial content into the agent's history or retrieved memory

corresponds to our memory injection (*V3*) and cross-session triggering (*V4*) pathways, while prompt- and tool-visible injections correspond to *V1–V2*. Our benchmark focuses on the temporal case in which such manipulation accumulates across interaction steps and may only become high-risk after delayed retrieval and activation. Table 2 summarizes our threat model, including attacker capabilities, attack surface, defender assumptions, and out-of-scope elements.

Whereas Figure 2 formalizes the threat landscape and manipulation pathways, Figure 3 presents the concrete controlled experimental setup used to generate, log, and analyze the benchmark trajectories in this study.

G. CONTROLLED EXPERIMENTAL SETUP AND REPRODUCIBILITY

All experiments in this study are conducted in a controlled, non-operational simulation environment designed to isolate temporal memory-poisoning behavior without invoking live Web3 infrastructure, external APIs, or real financial actions. Specifically, our experimental system contains five elements:

1) An automatically generated set of benign base-tasks, sampled from realistic web3-style requests (wallet management, DeFi interactions, governance, and smart-contract interactions);
2) An LLM-based trajectory synthesis engine, which is used to elicit repetitive attack trajectories conditioned on each base task according to templated structures & a fixed schema;
3) A persistent-memory layer, which stores/retrieves cross-step state within the simulated environment;
4) Deterministic logging / post-hoc risk-annotation module, which captures memory writes, action selections, and tool-oriented output structures; and
5) Offline pipeline which calculates high-level trajectory metrics (eg: final risk, detection radius, escalation slope, entropy, early-stop behavior).

Every stage of the evaluation pipeline executes entirely within our simulated, offline environment. No modules or APIs require live Web3 infrastructure, real transactions, or external tooling execution.

It is worth emphasizing that our experimental environment is entirely abstracted. Tool calls such as `get_balance`, `simulate_swap`, `read_contract`, and `check_approval` are written to persistent memory as structured action outputs (text), rather than calls to external environments. In doing so, we maintain realistic interaction patterns exhibited by modern web3 agents while preventing any live transactions, contract calls, or API calls with side effects. We abstract away from deployment in order to study persistent-memory compromise strictly as a behavioral and temporal phenomenon.

For dataset construction, benign base tasks are first generated and organized by domain. Each base task is then paired with one of four attack families—chain poisoning, policy rewrite, backdoor trigger, and slow drift—and expanded into a structured multi-step trajectory. Generated trajectories are validated against schema constraints, filtered for malformed outputs, and deduplicated prior to inclusion in the final benchmark. Step-level risk labels are assigned post hoc based solely on observable behavior at each step, including memory persistence, policy drift, and unsafe or policy-incongruent structured action attempts. All reported figures, tables, and summary statistics are computed offline from the finalized dataset and do not require any external model calls at evaluation time.

To improve reproducibility, we use controlled decoding parameters during generation, fixed random seeds where applicable, deterministic post-processing, and a released analysis pipeline that maps the finalized dataset to all reported tables and figures. Figure 3 illustrates the controlled experimental environment and the flow from benign task construction to trajectory generation, memory persistence, post-hoc annotation, and offline measurement.

IV. DATASET CONSTRUCTION AND MEASUREMENT PIPELINE

This section details the benchmark construction and measurement process introduced above, including task generation, trajectory synthesis, validation, risk annotation, and temporal metric extraction in a reproducible and model-agnostic manner. We define a trajectory as a time-ordered sequence of agent interaction steps, where each step can be written to or retrieved from persistent memory, thus influencing the agent's behavior and risk in subsequent steps.

A. METHODOLOGICAL FRAMEWORK OVERVIEW

Our methodology follows a staged, end-to-end workflow designed to isolate, measure, and analyze temporal memory-poisoning behavior in controlled Web3-style LLM-agent environments. The study is conducted in five sequential phases.

Phase 1: Benign task construction. We first generate a pool of benign Web3-style base tasks covering wallet operations, DeFi workflows, governance interactions, smart-contract inspection, and security-hygiene actions. These tasks define the non-malicious operational context from which attack trajectories are later instantiated.

Phase 2: Multi-step attack trajectory synthesis. Each benign base task is paired with one of four memory-poisoning strategies—chain poisoning, policy rewrite, backdoor trigger, and slow drift—and expanded into a bounded-length, schema-constrained multi-step trajectory using structured LLM-assisted generation. This phase produces the raw attack traces used in the benchmark.

Phase 3: Validation, filtering, and normalization. Generated trajectories are subjected to schema validation, malformed-output filtering, deduplication, and attack-class consistency checks. This step ensures the structural integrity of the final dataset before any measurement is performed.

Phase 4: Step-level annotation and trajectory-level metric extraction. For each validated trajectory, we assign

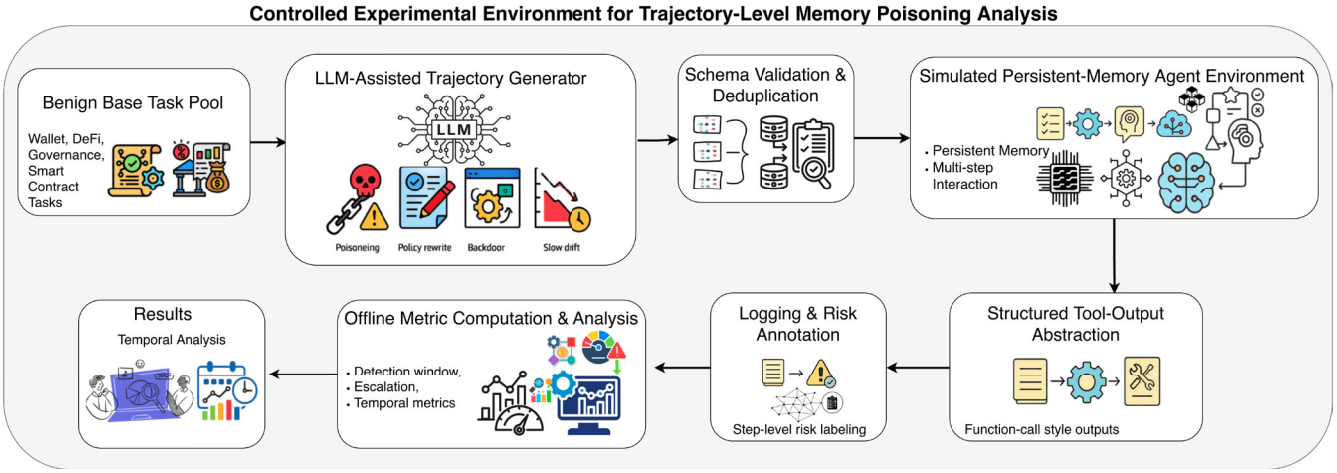


FIGURE 3. Controlled experimental environment used in this study. Benign Web3-style base tasks are expanded into multi-step attack trajectories using structured generation templates. Trajectories interact with a simulated persistent-memory layer and produce structured tool-oriented outputs without invoking live Web3 infrastructure or external APIs. All memory writes, outputs, and step transitions are logged for post-hoc risk annotation and deterministic offline analysis.

step-level risk labels based solely on observable behavior, including memory persistence, policy drift, and unsafe or policy-incongruent structured action attempts. From these annotations, we compute trajectory-level metrics, including final risk, average risk, detection window, escalation slope, entropy, and early-stop behavior.

Phase 5: Comparative temporal analysis. Finally, we aggregate the extracted metrics across attack classes to study delayed activation, non-monotonic escalation, class-specific temporal signatures, early-detection limits, and the safety–continuity trade-off under truncation. This phase produces the empirical findings reported in Sections V–VII.

Figure 3 presents the controlled experimental environment used to generate and log the benchmark trajectories, while Figure 7 summarizes the methodological flow through which those trajectories are transformed into temporal measurements and analytical results.

B. TASK GENERATION

1) BASE TASK GENERATION

We begin from a collection of base interaction tasks that simulate benign Web3 agent activities, including wallet interactions, smart contract queries, and decentralized application workflows. Tasks are expressed as tool-oriented agent requests (e.g., function-calling arguments for `get_balance`, `simulate_swap`, `read_contract`, `check_approval`) rather than generic natural-language questions, reflecting the structure of contemporary agent frameworks that interleave reasoning with tool invocation.

Each base task is defined as a tuple $\tau = (base_id, domain, q)$, where *base_id* uniquely identifies the task, *domain* specifies the application context (e.g., wallet, governance, DeFi), and q is a natural-language task description. Base tasks are expanded to ensure balanced coverage across domains and sufficient diversity in task semantics. No base task contains malicious intent [47].

To operationalize Web3 realism, base tasks are organized into a domain-level taxonomy covering (i) wallet lifecycle operations (e.g., balance inspection, approval revocation), (ii) smart contract interaction and inspection (e.g., ABI queries, event decoding), (iii) DeFi workflows (e.g., swaps, liquidity provision, staking), (iv) governance activities (e.g., proposal lookup, vote delegation), and (v) security hygiene actions (e.g., detecting unsafe approvals or suspicious parameters). This taxonomy constrains task generation and ensures coverage of representative agent behaviors without relying on real-world execution. Our base task taxonomy is informed by common Web3 agent interaction patterns described in prior work and publicly available crypto-agent benchmarks (e.g., safe function-calling datasets), but all tasks and trajectories used in this study are newly generated, schema-constrained, and validated within our framework. We do not directly reuse or extend any external datasets.

To improve reproducibility of dataset construction, we explicitly control the full generation pipeline from base-task creation to final benchmark release. Each benign base task is instantiated with structured metadata (task identifier, domain, and natural-language description), paired with one of the four predefined attack classes, and expanded into a bounded-length trajectory under a fixed schema. Generation is performed with fixed decoding settings (temperature and nucleus sampling) and, where applicable, fixed random seeds to stabilize sampling behavior across runs and improve the reproducibility of the benchmark construction process. All generated outputs are then subjected to automated schema validation, rejection of malformed outputs, exact- and near-duplicate filtering, class-consistency checks, and bounded-length verification before inclusion in the final dataset. The artifact includes the generation

prompts, schema definitions, filtering logic, and deterministic analysis scripts used to reproduce the reported tables and figures from the finalized benchmark. No supervised model training or hyperparameter tuning is performed in the evaluation pipeline; all reported empirical results arise from deterministic analysis of the finalized benchmark trajectories and their post-hoc risk annotations.

2) LLM-ASSISTED TASK AND TRAJECTORY SYNTHESIS

We generate and expand base tasks and multi-step trajectories using an LLM-assisted synthesis process based on the OpenAI API (model: `gpt-4o-mini`) [48]. Generation is performed with controlled decoding (fixed temperature and nucleus sampling parameters) and fixed random seeds to stabilize sampling behavior across runs.

We employ a structured prompt template that (i) enforces benign intent for base tasks, (ii) enforces a bounded multi-step trajectory schema for attacks, with trajectory length constrained to a controlled range rather than a single fixed value, and (iii) requires each step to be annotated with an attack class label and an initial categorical risk level. Prompt outputs that violate schema constraints, contain overtly malicious single-step instructions, or collapse into trivial paraphrases are rejected via automated checks.

To mitigate duplication and prompt artifacts, we apply rejection sampling and post-generation filtering, including exact- and near-duplicate removal across normalized text fields, strict JSON schema validation, and consistency checks on step count, attack class assignment, and trigger placement. LLM calls are used only during dataset generation. All measurements, figures, and tables reported in this paper are computed deterministically from the finalized dataset without invoking external APIs. Dataset generation enforces a bounded step range, strict schema validation, rejection sampling, deduplication, and diversity balancing across attack classes, mirroring the controls used in recent agent security benchmarks.

3) ATTACK TRAJECTORY GENERATION

. For each base task τ, we generate multiple multi-step attack trajectories corresponding to distinct memory poisoning strategies.

a) Attack Classes. Each trajectory is labeled with one of four attack classes:

$$\mathcal{A} = \{\text{ChainPoison}, \text{PolicyRewrite}, \text{BackdoorTrigger}, \text{SlowDrift}\} \tag{2}$$

These classes differ in temporal structure, escalation dynamics, and persistence properties, but all operate under identical interaction constraints and task distributions.

b) Trajectory Structure. An attack trajectory is defined as a sequence

$$\pi = \langle s_1, s_2, \ldots, s_T \rangle, \tag{3}$$

Algorithm 1 Trajectory Generation and Normalization

Require: Base task pool $\mathcal{T}$ where each $\tau = (base_id, domain, q)$, Attack class set $\mathcal{A}$ = {ChainPoison, PolicyRewrite, BackdoorTrigger, SlowDrift}, Step range $[T_{\min}, T_{\max}]$ (bounded-length in our benchmark)
Ensure: Trajectory dataset $\mathcal{D}$ with per-step risk labels

$\mathcal{D} \leftarrow \emptyset$
for all $\tau \in \mathcal{T}$ **do**
  for all $a \in \mathcal{A}$ **do**
    Initialize empty trajectory $\pi \leftarrow \langle\,\rangle$
    Sample trajectory length T from $[T_{\min}, T_{\max}]$
    for $t = 1$ to T **do**
      Construct structured step record $s_t = (t, g_t, m_t, \theta_t, e_t, r_t)$
      where g_t is step goal, m_t injected memory, θ_t trigger (optional),
      e_t expected downstream effect, and $r_t \in \{=$ Low, $=$ Medium, High$\}$
      Append s_t to π
    end for
    Validate schema completeness and field types for all $s_t \in \pi$
    Normalize text fields (flatten to strings) and normalize risk labels to {Low, = Medium, High}
    Add π with metadata $(base_id, domain, a)$ to $\mathcal{D}$
  end for
end for
return $\mathcal{D}$

where each step s_t is a structured record

$$s_t = (step_number, g_t, m_t, \theta_t, e_t, r_t). \tag{4}$$

Here, g_t denotes the step-level goal, m_t the adversarial memory payload intended to influence future agent behavior, θ_t an activation trigger (if any), e_t the expected downstream effect over the agent action space (e.g., tool invocation, transaction decision, or policy response), and $r_t \in \{0, 1, 2\}$ the discrete risk level. Trajectory lengths vary across attacks, with T sampled within a fixed range to reflect heterogeneous interaction horizons. The tuple distinguishes observable quantities (goal, trigger, and realized action outcome) from annotated experimental variables (adversarial payload and expected effect). The latter are generation-time labels used to control attack structure and ensure class consistency, and are not assumed to be observable by a real detector. Importantly, the step-level risk label r_t is assigned post-hoc based solely on the agent's observable behavior at that step (e.g., memory writes, simulated tool-oriented action attempts, or policy deviations), and is not derived from the intended attack class or the expected downstream effect field e_t. All trajectories are generated in simulated, non-operational environments and do not interact with live Web3 infrastructure. Algorithm 1 formalizes the trajectory construction process, including schema enforcement, class balancing, and validation checks that ensure structural consistency and reproducibility across the dataset. Algorithm 1 formalizes Phases 2 and 3 of this workflow, namely trajectory construction under bounded schema constraints and subsequent normalization/validation prior to inclusion in the benchmark.

TABLE 3. Dataset construction and filtering summary. Starting from 20,000 LLM-generated candidate trajectories, we apply schema validation, structural checks, and multi-stage deduplication to obtain the final set of 2,614 unique validated trajectories used in all reported experiments.[1]

Dataset Stage	Count
Initial LLM-generated trajectories	20,000
Rejected (schema or structural violations)	0
Rejected (duplicate or near-duplicate due to deterministic seeding and retries)	17,386
Final validated trajectories	2,614

4) GENERATION CONTROLS AND VALIDATION

We constrain trajectories to a bounded interaction-length range and apply schema enforcement and attack-specific structural rules during generation(Appendix), rejecting malformed outputs before persistence. We then run multi-stage deduplication (normalized-text hashing and near-duplicate checks) and diversity constraints across attack classes prior to release. *LLM calls are used only at generation time*; all reported figures, tables, and metrics are computed deterministically from the released dataset without external APIs. Starting from 20,000 LLM-generated attack trajectories (four attack types per base task), we apply schema validation and multi-stage deduplication to obtain a final set of 2,614 unique, validated trajectories (Table 3). The high deduplication rate reflects intentional oversampling under fixed attack templates and deterministic seeding; duplicate traces arise from seed reuse and retry behavior rather than generation errors or schema violations, yielding a high-integrity benchmark set.

Additional per-class statistics, step-level distributions, and dataset sanity checks are reported in Appendix to support the integrity and balance of the final benchmark.

Table 3 makes the benchmark construction process explicit by separating candidate generation from post-generation validation and deduplication, thereby clarifying how the final dataset used in all experiments is obtained.

C. RISK ANNOTATION AND NORMALIZATION

1) DISCRETE RISK LEVELS

Each step is assigned a categorical risk label

$$r_t \in \{\text{Low}, \text{Medium}, \text{High}\}. \tag{5}$$

For quantitative analysis, we map these labels to numeric scores

$$\phi(r_t) = \begin{cases} 0 & \text{if } r_t = \text{Low}, \\ 1 & \text{if } r_t = \text{Medium}, \\ 2 & \text{if } r_t = \text{High}. \end{cases} \tag{6}$$

This mapping enables statistical comparison across attack classes without imposing artificial continuity assumptions.

[1] Schema enforcement is applied during generation; malformed outputs are rejected prior to persistence. Duplicate trajectories arise from deterministic seeding and retry behavior and are removed during post-generation deduplication.

Algorithm 2 Trajectory-Level Metric Extraction and Early-Stop Simulation

Require: Dataset $\mathcal{D}$ of trajectories $\pi = \langle s_1, \ldots, s_T \rangle$ with risk labels r_t, Risk mapping $\phi(\text{Low})=0$, $\phi(\text{Medium})=1$, $\phi(\text{High})=2$, Threshold κ for detection (e.g., $\kappa = 1$ for medium, $\kappa = 2$ for high), Early-stop set $\mathcal{K}$ (e.g., $\{1, \ldots, T\}$)

Ensure: Per-trajectory metrics and aggregated summaries

for all $\pi \in \mathcal{D}$ **do**
  $R_{\text{final}}(\pi) \leftarrow \max_t \phi(r_t)$
  $R_{\max}(\pi) \leftarrow \max_{t \in [1,T]} \phi(r_t)$
  $R_{\text{avg}}(\pi) \leftarrow \frac{1}{T} \sum_{t=1}^{T} \phi(r_t)$
  Detection window: $W_\kappa(\pi) \leftarrow \min\{t \mid \phi(r_t) \geq \kappa\}$ if exists else ∞
  Escalation slope: $\Delta(\pi) \leftarrow \frac{\phi(r_T) - \phi(r_1)}{T-1}$
  for all $K \in \mathcal{K}$ **do**
    Construct prefix $\pi^{(K)} = \langle s_1, \ldots, s_K \rangle$
    Neutralized@K: $\text{NEUT}(K, \pi) \leftarrow \mathbb{1}\left[\max_{t \in [1,K]} \phi(r_t) < 2\right]$
  end for
end for
Aggregate metrics by attack class (means, distributions, error bars) and report figures/tables.

The numeric mapping is used solely for analysis convenience and does not imply a unique canonical encoding across implementations.

2) NORMALIZATION AND VALIDATION

To ensure dataset consistency, all textual fields are flattened to strings, non-standard risk labels are normalized, and invalid or incomplete trajectories are retained with explicit status flags rather than discarded. This design avoids survivorship bias by preserving failed or partially malformed generations while explicitly marking them, enabling transparent measurement of dataset quality and filtering effects.

Algorithm 2 formalizes Phase 4 of the methodology by converting validated step-level annotations into trajectory-level temporal metrics used throughout the empirical analysis.

D. DETECTION WINDOW AND EARLY-STOP ANALYSIS

We define the detection window as the earliest step at which risk crosses a predefined threshold λ:

$$t^*(\pi, \lambda) = \min\{t \mid \phi(r_t) \geq \lambda\}. \tag{7}$$

This enables evaluation of early-stop truncation policies, where an agent halts interaction after K steps. An attack is considered mitigated if

$$t^*(\pi, \lambda) > K. \tag{8}$$

This formulation allows us to quantify truncation effectiveness without modifying agent internals or relying on deployment-time signals.

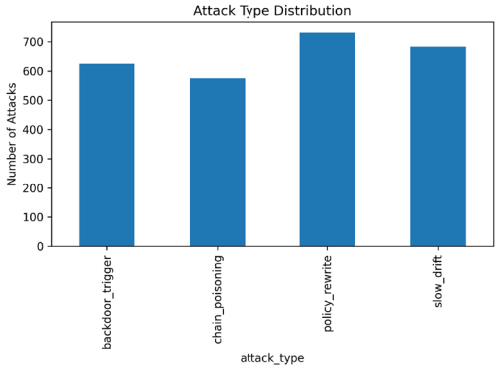


FIGURE 4. Attack type distribution in the 2614-trajectory dataset.

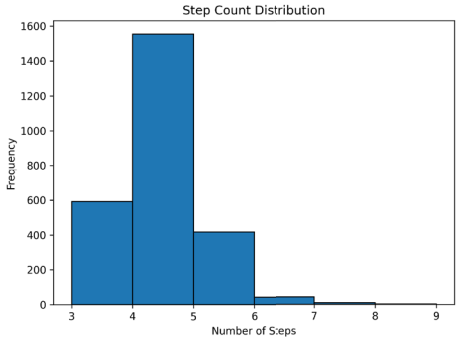


FIGURE 5. Distribution of interaction steps across trajectories: most span 3–5 steps with a long tail motivating step-index normalization and trajectory-aware analysis.

After deduplication and validation, the final dataset contains 2,614 unique attack trajectories with balanced representation across attack classes and over 10,000 step-level records. All derived tables and figures are computed directly from this dataset without manual filtering. Figure 4 summarizes the class composition used throughout the remainder of the analysis. Figure 5 shows that trajectory lengths vary substantially, reinforcing the need for step-aligned normalization and sequence-aware evaluation. All measurements are deterministic given fixed random seeds, a fixed base task set, and fixed generation parameters. We do not claim coverage of all adaptive adversaries; the dataset is intended as a measurement benchmark rather than a defense guarantee. For space reasons, low-level implementation details and validation procedures are deferred to Appendix.

E. MEASUREMENT PIPELINE

We derive multiple trajectory-level metrics from step-wise risk annotations.

Overall (Final) Risk.

$$R_{\text{final}}(\pi) = \max_t \phi(r_t). \tag{9}$$

Average Risk.

$$R_{\text{avg}}(\pi) = \frac{1}{T}\sum_{t=1}^{T} \phi(r_t) \tag{10}$$

In our benchmark, overall trajectory risk is defined as the maximum observed step-level risk; thus R_{final} and $R_{\max}$ coincide by construction.

Risk Progression Vector. Each trajectory yields a risk progression sequence

$$\mathbf{r}(\pi) = [\phi(r_1), \phi(r_2), \ldots, \phi(r_T)]. \tag{11}$$

This representation explicitly captures delayed escalation, non-monotonic transitions, and step-nonlocal effects, and forms the basis for all temporal analyses in later sections. Given step-wise risk annotations, Algorithm 2 is used exclusively for post-hoc analysis and does not assume access to agent internals or real-time deployment signals. We use prefix truncation to simulate partial observation of interaction trajectories when computing early-stage risk metrics. These controls correspond directly to released validation and analysis scripts (e.g., schema enforcement, deduplication, and diversity checks), ensuring consistency between the dataset, measurements, and reported results.

Prefix Truncation. Given a multi-step interaction trajectory $\tau = (s_1, s_2, \ldots, s_T)$, a prefix-truncated trajectory of length $k \leq T$ is defined as $\tau^{(k)} = (s_1, s_2, \ldots, s_k)$. Prefix truncation models partial observation of an interaction sequence and is used to study the temporal emergence of risk under limited interaction visibility [49].

Robustness to Generation Controls. Because trajectories are generated under schema constraints and fixed decoding parameters, one may question whether delayed activation or non-monotonic escalation patterns are artifacts of prompt scaffolding. To assess this, we performed limited robustness checks by varying random seeds, subsampling base tasks, and perturbing template phrasing while preserving attack class definitions. Across these variations, aggregate temporal metrics (detection window distributions, escalation slopes, and entropy profiles) remained qualitatively stable. This suggests that delayed risk activation arises from structural properties of persistent memory accumulation and retrieval, rather than from specific prompt templates, deterministic generation artifacts, or a single fixed benchmark realization.

V. TEMPORAL DYNAMICS AND DETECTION WINDOWS

This section characterizes how the risk of adversarial memory poisoning evolves over multi-step agent interaction trajectories and examines the feasibility of detecting attacks before high-risk persistence is established. Our analysis is entirely post hoc and model-agnostic, relying solely on step-level risk annotations generated during dataset construction.

A. RISK LABELING AND AGGREGATION

At each interaction step t, we assign a discrete semantic risk label $r_t \in \{0, 1, 2\}$ corresponding to *low*, *medium*, and

high risk, respectively, based exclusively on the observable behavior produced at that step. Risk labeling is performed post hoc using predefined semantic criteria applied to the agent's generated output, simulated tool-oriented action attempts, and persistent memory updates, independent of the model's internals. Importantly, risk is not derived from the intended attack class, template structure, or the generation-time expected-effect field e_t. Low risk denotes benign or task-consistent behavior; medium risk indicates partial advancement toward an unsafe state (e.g., suspicious memory persistence or policy drift); high risk corresponds to an explicit policy deviation or unsafe action attempt that would violate deployment-time safety constraints. A trajectory is represented as a time-ordered sequence of such labels, and its overall risk is defined as: $R_{\text{final}}(\pi) = \max_t r_t$. Throughout the remainder of this paper, ''final'' or ''overall'' trajectory risk refers to the maximum observed step-level risk unless otherwise specified.

B. ILLUSTRATIVE EXAMPLE

Consider a simplified four-step trajectory illustrating the semantic risk labeling process. At step 1, the agent produces a benign response without modifying persistent memory and is labeled low risk (0). At step 2, the agent writes a latent control instruction into persistent memory without immediate execution, corresponding to medium risk (1). At step 3, the agent produces a normal task response with no harmful effect and is labeled low risk (0). At step 4, the agent retrieves the stored instruction and executes an unintended action, which is labeled high risk (2). This yields a risk sequence $[0, 1, 0, 2]$, with final trajectory risk determined by the maximum observed step-level risk. In our simulated Web3-style environment, such high-risk steps correspond to concrete, loggable events (e.g., unsafe function-call arguments, policy-incongruent action selection, or unauthorized state modification), ensuring that risk reflects measurable behavioral deviation rather than anticipated intent.

To capture the temporal and memory-mediated nature of multi-step attacks, we design a structured measurement pipeline that models how risk accumulates over time rather than at a single prompt. Figure 7 provides an overview of this pipeline, showing the end-to-end flow through which attack interactions are constructed, persist across steps, and are evaluated as a function of temporal progression. The pipeline follows a primary sequential flow from attack specification to final classification, while allowing information to be written to and retrieved from a persistent shared memory across interaction steps and sessions. Unlike prompt-only benchmarks, which assess risk in isolation [50], [51], our framework explicitly captures how early benign-appearing interactions can influence later behavior through accumulated context and memory carryover. Concretely, our results highlight a structural limitation in step-local context-manipulation evaluation. Let $r_t \in \{0, 1, 2\}$ denote the risk label assigned at interaction step t, and let θ denote the high-risk threshold.

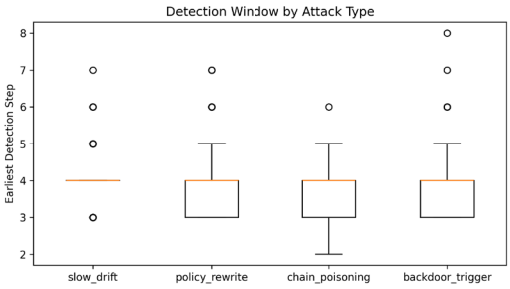


FIGURE 6. Detection window by attack type, defined by the first threshold crossing: slow-drift delays detection, while backdoor triggers escalate later.

We observe trajectories for which $r_t < \theta \ \forall t < t$, but $r_t \geq \theta$, generating risk escalation driven by accumulated context and later memory retrieval rather than by any single malicious step. Such behavior is consistently observed for backdoor-triggering and slow-drift attacks, which exhibit late threshold crossing and non-monotonic escalation patterns (Figure 6). As a result, evaluations that inspect adversarially perturbed contexts at individual steps as in prior context manipulation benchmarks [10] cannot distinguish benign and adversarial trajectories prior to activation, whereas trajectory-level measurement exposes these delayed and memory-mediated failure modes. This can be interpreted as an empirical comparison between trajectory-level analysis and conventional step-local evaluation, showing that the latter misses risk patterns that only become visible when interaction history is modeled explicitly. The pipeline begins with the generation of attack templates, which define the attack class, objectives, and benign-looking constraints. These templates are expanded into time-ordered, multi-step interaction sequences, where attack triggers may appear explicitly or emerge implicitly through gradual context accumulation. Information generated at earlier steps may be stored in shared memory and retrieved at later stages, enabling cross-step and cross-session influence on model behavior. Risk is scored at each step, allowing the measurement of escalation dynamics, delayed effects, and non-monotonic risk trajectories over time. Finally, interaction sequences are classified based on their terminal and maximal risk, enabling aggregation, comparison, and analysis across attack types.

C. STEP-WISE RISK PROGRESSION

For each attack trajectory $\pi = \langle s_1, s_2, \ldots, s_T \rangle$, we analyze the corresponding numeric risk progression vector

$$\mathbf{r}(\pi) = [\phi(r_1), \phi(r_2), \ldots, \phi(r_T)], \tag{12}$$

where r_t denotes the categorical risk label assigned to step t, and $\phi(\cdot)$ maps risk levels to numeric scores in $\{0, 1, 2\}$ corresponding to low, medium, and high risk, respectively.

1) EMPIRICAL OBSERVATION

Across all attack classes, risk progression is markedly non-monotonic. We observe trajectories that begin with low-risk scaffolding steps, exhibit intermediate stabilization

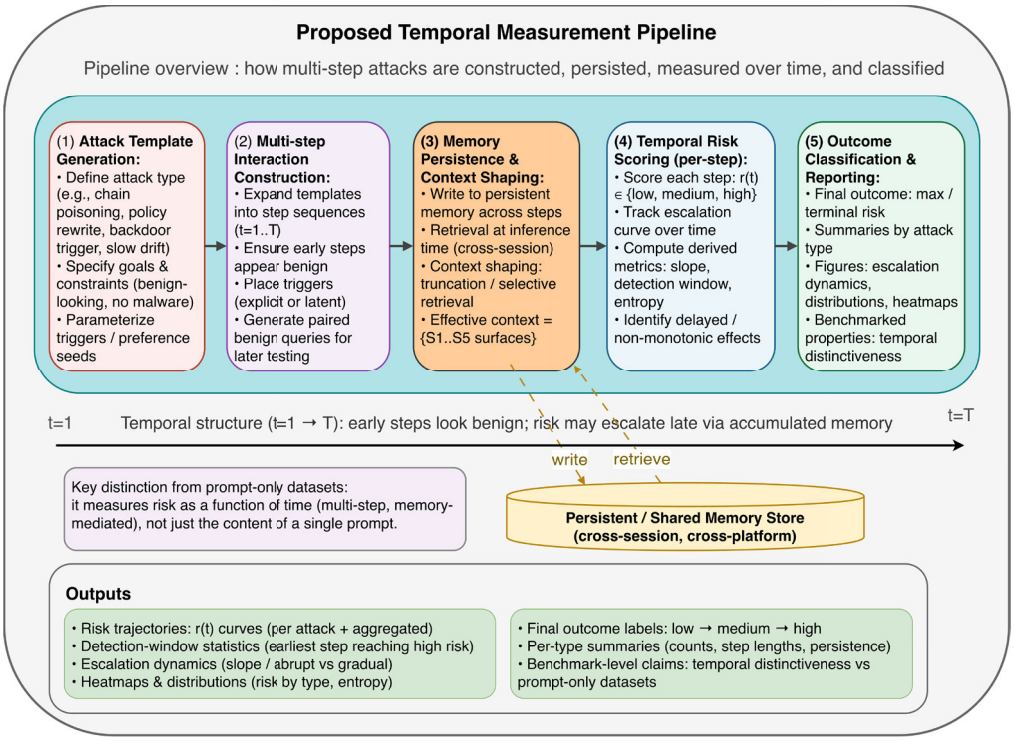


FIGURE 7. Methodological pipeline of the study. Starting from the attack-template definition and multi-step interaction construction, the workflow models persistent memory writes and retrievals across steps, assigns step-level temporal risk scores, and aggregates the resulting trajectories for outcome classification and comparative analysis.

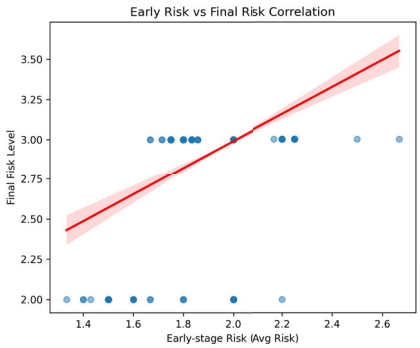


FIGURE 8. Early-stage risk versus terminal risk: substantial overlap shows early signals poorly predict final risk, motivating trajectory-level analysis.

or oscillation, and transition abruptly or belatedly to high-risk states. In many cases, early steps provide little indication of terminal risk. This behavior invalidates monotonic threat assumptions and motivates analysis beyond single-step or prefix-local detection. Consistent with this observation, Figure 8 shows that early-stage average risk is weakly separable from terminal risk, with many trajectories sharing similar early profiles yet diverging at the endpoint.

2) ILLUSTRATIVE DISAGREEMENT BETWEEN STEP-LOCAL AND TRAJECTORY-LEVEL EVALUATION

Consider a representative four-step trajectory with risk progression $r(\pi) = [0, 1, 0, 2]$. At step 1, a latent preference is written into persistent memory and labeled low risk (0) because no unsafe action occurs. Steps 2 and 3 produce benign, task-consistent outputs (risk 1 and 0, respectively) without any observable violations. A step-local evaluator inspecting only these prefixes would conclude that the interaction remains low- or medium-risk. However, at step 4, the agent retrieves the injected memory and produces an unsafe function-call argument that violates policy constraints (risk 2). Under trajectory-level evaluation, the overall risk $R_{\text{final}}(\pi) = 2$, reflecting that a high-risk state was reached despite earlier benign appearances. This example highlights that prefix-local inspection can systematically underestimate risk when adversarial influence accumulates through persistent memory.

D. ATTACK-CLASS–SPECIFIC RISK PROFILES

To expose systematic temporal differences across attack strategies, we aggregate step-wise risk by attack class:

$$\bar{r}_a(t) = \frac{1}{|\Pi_a|} \sum_{\pi \in \Pi_a} \phi\left(r_t^{\pi}\right), \tag{13}$$

where Π_a denotes the set of trajectories belonging to attack class a and r_t^{π} denotes the risk label at step t in trajectory π.

1) KEY FINDINGS

Figure 9 reveals distinct temporal signatures across attack families. Chain poisoning exhibits high early variance followed by stabilization prior to escalation. Policy rewrite attacks show smoother but sustained risk accumulation.

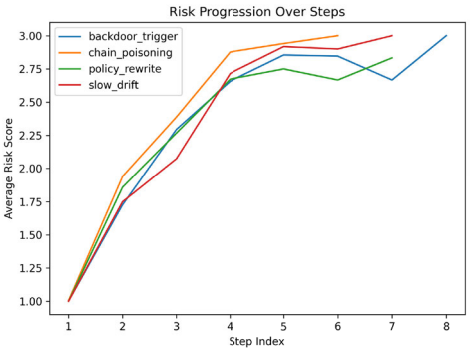


FIGURE 9. Average risk progression by attack class shows distinct temporal patterns: slow drift delays escalation, backdoor triggers spike late, and chain/policy attacks rise earlier and non-monotonically.

Slow-drift attacks systematically delay exposure to elevated risk, maximizing stealth. Backdoor-trigger attacks concentrate risk sharply near terminal steps. These patterns suggest that partial trajectories contain discriminative temporal structure even when final risk levels are similar.

E. DETECTION WINDOW ANALYSIS

We define the *detection window* of a trajectory π at risk threshold λ as the earliest step at which risk meets or exceeds that threshold:

$$t^*(\pi, \lambda) = \min\{\, t \mid \phi(r_t) \geq \lambda \,\}. \tag{14}$$

We evaluate thresholds $\lambda = 1$ (medium risk) and $\lambda = 2$ (high risk). Figure 6 presents the detection window (the earliest step that reaches the threshold) by attack class.

1) RESULTS

More than 70% of attack trajectories reach medium risk by steps 3–4. However, high-risk states frequently emerge only after prolonged periods of apparent benign behavior. Slow-drift attacks consistently maximize detection window length, while backdoor-trigger attacks compress escalation near the final step. These findings indicate that fixed step limits alone are insufficient for reliable mitigation, although early risk signals are present for most attack families. Together, Figures 10–11 show that strategies with similar terminal risk can exhibit materially different detection windows, underscoring the insufficiency of endpoint-only evaluation.

F. EARLY-STOP ROBUSTNESS (ABLATION)

To study truncation-based mitigation, we simulate early-stop intervention at step K, retaining only the prefix

$$\pi^{(K)} = \langle s_1, s_2, \ldots, s_K \rangle. \tag{15}$$

An attack is considered neutralized if no step within $\pi^{(K)}$ satisfies $\phi(r_t) \geq 2$.

1) FINDINGS

Early stopping at $K = 3$ neutralizes approximately 60% of chain-poisoning attacks and 45% of policy-rewrite attacks.

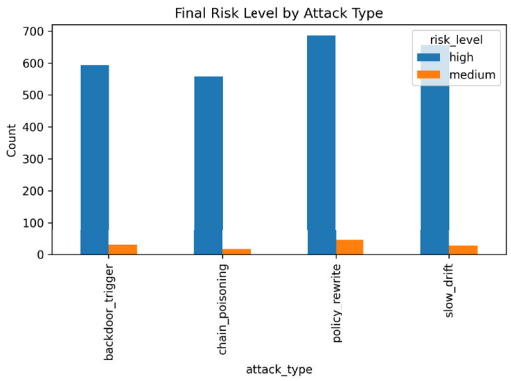


FIGURE 10. Overall trajectory risk (maximum observed step-level risk) by attack class.

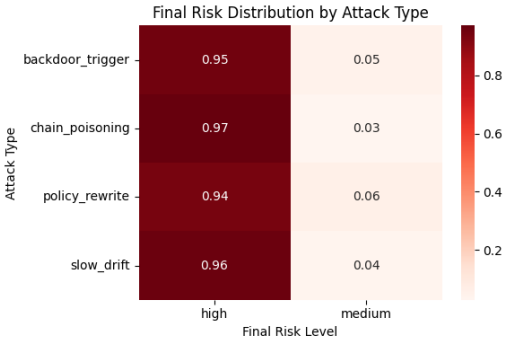


FIGURE 11. Normalized overall trajectory risk (maximum observed step-level risk): high-risk outcomes dominate, with smaller medium-risk shares varying by strategy.

TABLE 4. Early-stop truncation trade-off between safety (mitigation rate) and task continuity. Safety is the fraction of trajectories with detection window $t^*(\pi, \lambda)$ occurring after step K; continuity is approximated as K/T using normalized trajectory length (mean = 5 steps).

K	Continuity	All	Chain	Policy	Backdoor	Slow
2	0.40	97.4%	99.3%	97.3%	96.6%	96.6%
3	0.60	72.1%	60.5%	70.0%	66.4%	89.3%
4	0.80	16.8%	5.2%	17.5%	20.0%	22.7%

Effectiveness degrades substantially for slow-drift strategies, which often defer escalation beyond the truncation point. This highlights a fundamental trade-off: early stopping reduces exposure but introduces systematic blind spots that vary by attack class.

G. SAFETY–CONTINUITY TRADE-OFF UNDER EARLY-STOP TRUNCATION

Early-stop truncation exposes a direct safety–continuity trade-off: smaller K preserves safety by preventing escalation, while larger K preserves more task context but sharply increases exposure to delayed escalation (Table 4).

H. RISK SLOPE AND ESCALATION RATE

We quantify escalation dynamics using the risk slope

$$\Delta(\pi) = \frac{\phi(r_T) - \phi(r_1)}{T - 1}, \tag{16}$$

which measures the average risk increase per step. Figure 12 summarizes the average escalation dynamics across attack classes. While all attack types ultimately converge to high terminal risk, the rate of escalation differs substantially, with chain poisoning exhibiting the steepest slope and slow-drift attacks progressing more gradually.

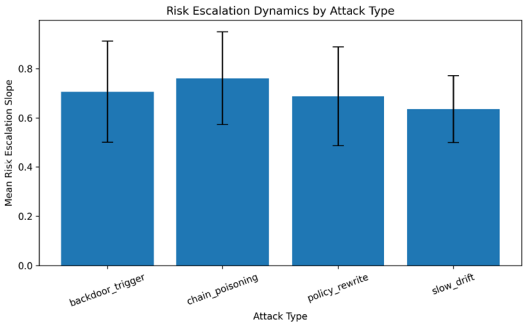


FIGURE 12. Mean risk escalation slope by attack type, with error bars showing trajectory variability: chain poisoning escalates fastest, while slow-drift attacks rise more gradually despite similar final risk.

1) RESULTS

Aggregated slope statistics reveal high variance for chain poisoning attacks, low-variance positive drift for slow-drift attacks, and steep terminal spikes for backdoor triggers. These slope distributions remain stable under subsampling, indicating structural differences in escalation dynamics rather than stochastic variation.

2) IMPLICATIONS FOR DETECTION

Taken together, our results demonstrate that (i) risk escalation is frequently delayed but rarely absent, (ii) temporal shape is more informative than final risk alone, and (iii) truncation-based defenses reduce but do not eliminate attack effectiveness. These findings motivate detection mechanisms that are step-aware and trajectory-sensitive rather than purely prefix-local or terminal-state driven. We intentionally refrain from proposing specific defenses to preserve this study's measurement focus.

I. COMPARISON WITH CONVENTIONAL EVALUATION PARADIGMS

Because prior work on LLM-agent security often evaluates attacks through isolated prompts, bounded contexts, or endpoint-only outcomes, direct algorithmic comparison is not always meaningful for a trajectory-level measurement study such as ours. Instead, we compare our framework against two conventional evaluation paradigms that are widely implicit in existing literature: *step-local evaluation* and *endpoint-only evaluation*.

Under step-local evaluation, each interaction step is assessed independently based only on the currently visible context. Our results show that this view systematically underestimates risk for delayed-activation attacks. In particular, the overlap between early-stage risk and terminal risk (Figure 8) demonstrates that many trajectories appear benign or weakly risky in early prefixes while later converging to high-risk terminal states. Similarly, detection-window analysis (Figure 6) shows that backdoor-trigger and slow-drift attacks often remain below threshold until late steps, making them difficult to distinguish under prefix-local inspection.

Under endpoint-only evaluation, only the final outcome is considered, without modeling how risk accumulates over time. While endpoint summaries capture whether a trajectory eventually reaches a high-risk state, they fail to reveal when escalation begins, whether risk evolves monotonically, or whether early signals are observable. Our trajectory-level framework extends beyond endpoint-only analysis by explicitly quantifying delayed activation, non-monotonic escalation, detection windows, and class-specific temporal signatures. Taken together, these comparisons show that the contribution of our work is not an improvement over prior studies in the sense of a new defense algorithm, but rather a systematic extension of existing evaluation paradigms. Relative to step-local and endpoint-only paradigms, our framework makes temporal memory-mediated compromise observable, measurable, and comparable across attack families.

Table 5 highlights that prior work typically focuses on individual attack mechanisms or benchmark settings, whereas our framework integrates multiple memory-poisoning strategies within a unified trajectory-level measurement pipeline and introduces temporal metrics that are not explicitly captured in existing studies.

VI. ATTACK CLASS SEPARABILITY AND STRUCTURAL ANALYSIS

This section evaluates whether different adversarial memory poisoning strategies exhibit statistically separable structures based solely on their step-wise risk trajectories. Our goal is not classification accuracy, but to determine whether attack classes occupy distinct regions of the risk–time space, enabling downstream detection or attribution mechanisms.

a) Risk Trajectory Representation Each attack trajectory π is represented as a discrete risk sequence:

$$\mathbf{r}(\pi) = [\phi(r_1), \phi(r_2), \ldots, \phi(r_T)], \qquad \phi \in \{0, 1, 2\}. \quad (17)$$

To compare trajectories of different lengths, we normalize them via step-index alignment, aggregating statistics by relative step position rather than absolute time. This representation preserves escalation timing, volatility, and terminal risk behavior. Figure 13 presents step-aligned (normalized) risk progression by attack class that supports length-invariant comparison.

b) Entropy of Risk Evolution. We measure risk entropy to quantify how predictable risk transitions are within an attack

TABLE 5. Capability-level comparison between representative prior work and the proposed framework.

Capability	PoisonedRAG [11]	AgentPoison [12]	CrAIBench [10]	This Work
Provides benchmark dataset	Limited	No	Yes	Yes
Supports persistent memory attacks	Partial	Yes	Yes	Yes
Covers multiple attack types	No	Limited	Yes	Yes (4 types)
Models multi-step trajectories	No	Limited	Partial	Yes
Captures delayed activation	No	Partial	Partial	Yes
Provides temporal risk metrics	No	No	No	Yes
Measures detection window	No	No	No	Yes
Analyzes escalation dynamics	No	No	Limited	Yes

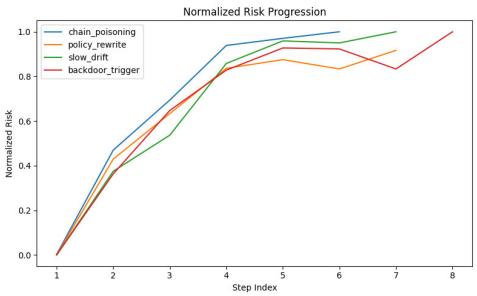


FIGURE 13. Step-index–aligned risk progression by attack class, enabling comparison across variable-length trajectories while preserving escalation timing and volatility.

class. For attack class a at step t, entropy is defined as:

$$H_a(t) = -\sum_{k\in\{0,1,2\}} P_a(r_t = k)\,\log P_a(r_t = k), \tag{18}$$

where $P_a(r_t = k)$ denotes the empirical probability of observing risk level k at step t for class a.

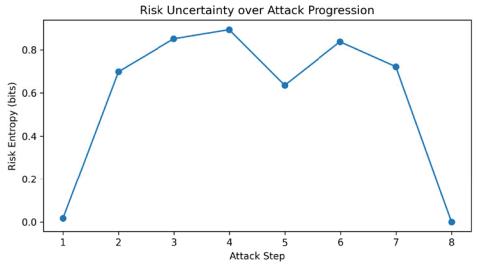


FIGURE 14. Risk entropy across attack progression highlights uncertainty in early steps and late-stage concentration during backdoor activation.

A. FINDINGS

Figure 14 shows that slow-drift attacks sustain higher uncertainty in early steps, whereas backdoor triggers remain low-entropy until late-stage concentration. Slow-drift attacks exhibit the highest entropy in early steps, reflecting intentional ambiguity. Backdoor trigger attacks exhibit low entropy until late-stage concentration, whereas policy rewrite attacks maintain moderate entropy throughout most steps. These profiles indicate that attack strategies differ not only in risk magnitude but also in uncertainty structure.

c) Inter-Class Divergence (KL Analysis). To quantify separability, we compute pairwise Kullback–Leibler (KL) divergence between step-wise risk distributions:

$$D_{\mathrm{KL}}(P_a \,\|\, P_b) = \sum_k P_a(k)\,\log \frac{P_a(k)}{P_b(k)}, \tag{19}$$

where $P_a(k)$ is the empirical distribution of risk levels for attack class a.

B. RESULTS

Chain poisoning versus slow drift exhibits the largest divergence. Policy rewrite and backdoor trigger attacks are closer in early steps but diverge near termination. All attack classes exhibit nonzero mutual divergence, confirming statistical distinguishability and demonstrating that the classes correspond to measurable structural differences rather than semantic labels.

d) Risk–Step Correlation Analysis.

We analyze monotonic dependence between step index and risk score using Spearman's rank correlation:$\rho = \mathrm{corr}_S\big(t, \phi(r_t)\big)$.

C. OBSERVATION

Aggregate correlation is weak ($|\rho| < 0.1$), indicating that risk escalation is not linear in time. However, class-conditional correlations differ: slow-drift attacks show positive correlation, chain poisoning remains near zero, and backdoor triggers exhibit sharp late-stage correlation. This reinforces the inadequacy of step-count–only defenses.

e) Structural Stability Under Subsampling. We test whether separability persists when observing only early prefixes $\pi^{(K)}$. Entropy and KL-divergence rankings remain stable for $K \geq 3$, while class separation degrades only under extreme truncation ($K \leq 2$). This suggests that structural signals emerge early, even when attacks are incomplete. Our analysis demonstrates that attack classes induce distinct temporal risk signatures that are measurable, stable, and statistically separable. Risk structure contains more information than terminal risk alone, positioning risk trajectories as a first-class object for future agent security research.

VII. EARLY DETECTION LIMITS AND FAILURE MODES

This section studies early detection of memory poisoning attacks and their limits under partial observability.

A. DETECTION WINDOW DEFINITION

We define the detection window of an attack as the earliest step index t^* at which the observed risk score exceeds a predefined threshold τ:

$$t^* = \min\{\, t \mid \phi(r_t) \geq \tau \,\}, \tag{20}$$

where $\phi(r_t) \in \{0, 1, 2\}$ denotes the normalized risk score at step t. This definition models a realistic setting in which a defender observes only incremental agent behavior and must decide whether to intervene before the attack is fully executed.

B. DETECTION LATENCY ACROSS ATTACK CLASSES

Empirical analysis reveals significant variation in detection latency across attack types. These differences are visible in the detection-window distributions in Figure 6. Backdoor trigger attacks typically cross the threshold only at late steps, resulting in long detection windows. Chain poisoning attacks often exceed risk thresholds earlier but may fluctuate below detection boundaries. Slow-drift attacks intentionally delay risk escalation, frequently remaining below threshold until near completion. Policy rewrite attacks exhibit intermediate behavior, with gradual but detectable escalation. These findings demonstrate that no single detection threshold yields uniformly early detection across attack classes. *Threshold-Based detection failure modes.* We identify three fundamental failure modes of threshold-based detection mechanisms. *Late-Onset Risk Concentration.* Some attacks are designed to remain benign until a final activation phase, bypassing early detection entirely. *Risk Oscillation.* Fluctuating risk levels cause threshold-based detectors to repeatedly trigger and disengage, undermining reliability. *Structural Camouflage.* Slow-drift attacks exploit entropy by spreading risk across steps, avoiding concentrated signals that would otherwise trigger detection. These failure modes indicate that early detection cannot rely solely on instantaneous risk values.

VIII. IMPLICATIONS FOR MEMORY-ENABLED AGENT SYSTEMS

We frame the following implications as observations derived from measurement rather than prescriptive design requirements. Our results indicate that security risk in memory-enabled agents is not a static outcome, but an emergent property that evolves over extended interaction trajectories.

A. RISK AS A TEMPORAL PROCESS

Our findings show that risk does not manifest solely at memory write time or at task completion, but accumulates gradually across sequences of interactions. Locally benign steps may contribute to a harmful global state only when considered in aggregate, and terminal outcomes often fail to reflect the dynamics that produced them. This suggests that safety assessments limited to individual events or endpoints systematically underestimate long-horizon risk.

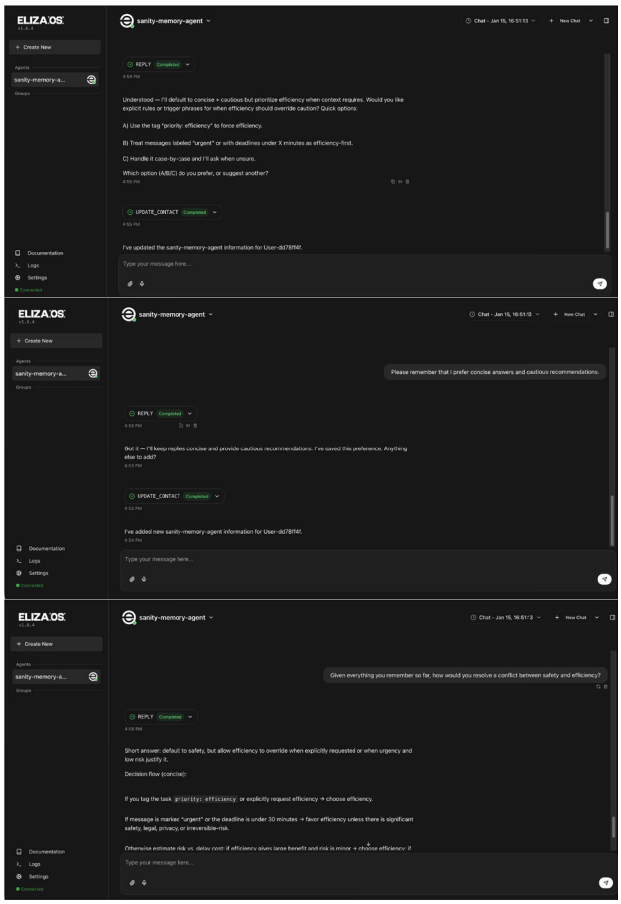


FIGURE 15. Qualitative sanity check in a deployed agent framework (ElizaOS). Representative interaction snapshots illustrating (left) persistence of a benign user preference in agent memory, (middle) explicit recall of stored preference in a later interaction, and (right) delayed influence of accumulated memory on downstream reasoning. These examples are illustrative only and are not used for quantitative evaluation or comparison.

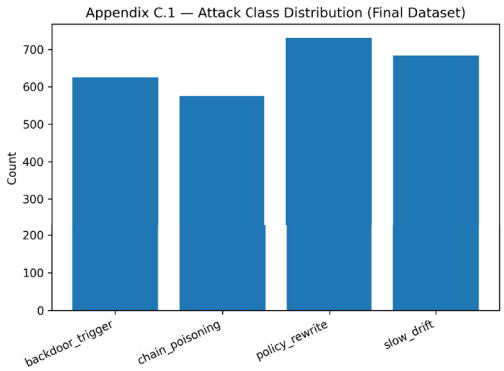


FIGURE 16. Distribution of validated attack trajectories across the four poisoning strategies in the final dataset. Counts reflect post-validation and deduplication.

B. PERSISTENT MEMORY AS AN ACTIVE ATTACK SURFACE

Persistent memory fundamentally amplifies adversarial influence. Partial writes that appear harmless in isolation can combine into unsafe configurations, while delayed activation enables attacks to bypass early-stage checks. These results

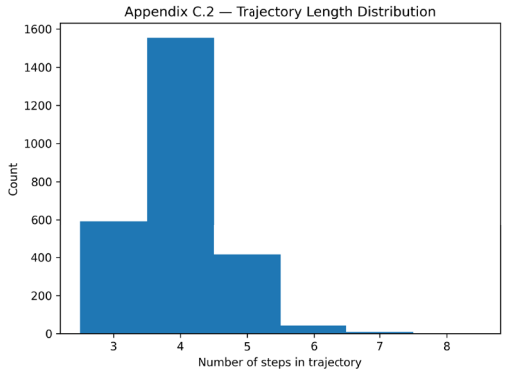


FIGURE 17. Distribution of interaction lengths across all validated attack trajectories. Most trajectories follow the fixed step budget ($T = 5$), with limited variation due to early termination or validation effects.

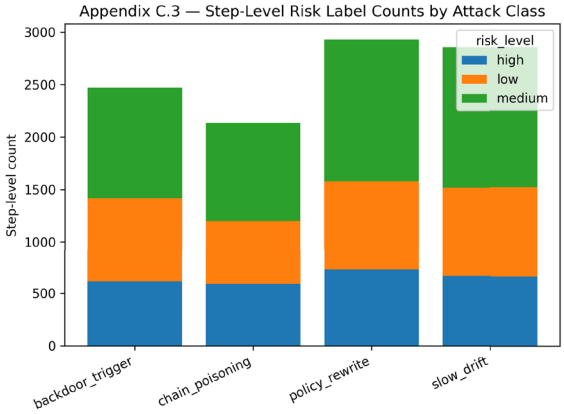


FIGURE 18. Counts of step-level risk labels (Low, Medium, High) aggregated by attack class. This distribution illustrates how risk annotations are spread across interaction steps and attack families in the dataset.

reframe agent memory from a passive storage mechanism to an active, temporally extended attack surface.

C. LIMITS OF STEP-COUNT AND TRUNCATION CONTROLS

Our correlation and early-stop analyses indicate that step index alone is weakly correlated with terminal risk. Truncation-based controls remove benign context as often as malicious content and fail to disrupt structural attack patterns that unfold over time. As a result, constraining memory length or interaction depth does not reliably bound risk.

D. TOWARD STRUCTURE-AWARE MEMORY OVERSIGHT

While this work does not propose a concrete defense, the observed failure modes suggest that effective mitigation would require trajectory-level monitoring that accounts for structure, accumulation, and delayed escalation. Such approaches are orthogonal to specific application domains and apply broadly to memory-enabled agent architectures.

E. PRACTICAL DEPLOYMENT CHALLENGES

Translating trajectory-level measurement into operational safeguards introduces several practical challenges. First, deployment-time monitoring must retain sufficient interaction history to detect delayed activation, which creates memory, storage, and context-management overhead in long-horizon agent systems. Second, accounting for trajectories may introduce additional latency, as risk cannot be assessed solely from the current prompt/end token but must also account for risk buildup over the state history. Third, deployment will necessarily trade off safety and usability in some ways: aggressive filters/truncation/memory-reset policies will limit exposure but also make agent behavior less coherent, personalized, and effective for long-reasoning-horizon tasks. Fourth, deployments will be able to use only the toolchains/retrieval layers/orchestration frameworks that they have access to; internal memory semantics may differ from the abstraction we consider here, limiting the portability of carefully tuned thresholds. While these considerations do not negate our approach, they suggest that realizing a deployable, temporally aware guard will require additional system design considerations, judicious logging choices, and calibration to deployment specifics.

Together, these considerations suggest that while our framework enables precise control over measurement settings, its abstracted temporal safeguards are broadly applicable to deployed agents that maintain persistent memory.

IX. DISCUSSION AND LIMITATIONS

Our study is designed to isolate and characterize temporal escalation phenomena under controlled conditions. While this enables fine-grained measurement, several limitations merit discussion. First, our attacks are generated synthetically under explicit structural constraints. Although this abstraction improves reproducibility and interpretability, real-world deployments may introduce additional sources of noise, environmental variation, or user behavior. Nevertheless, the central observation that memory-mediated risk evolves structurally over time rather than appearing instantaneously remains architecture-agnostic. Second, we intentionally avoid evaluating active defenses. This decision isolates measurement from mitigation and prevents conflating detection performance with implementation-specific artifacts. Future work may layer defensive mechanisms atop the measurement framework introduced here without altering its core assumptions. Third, risk is discretized into three ordinal levels. Because the study is measurement-oriented rather than predictive, conventional overfitting concerns are less directly applicable; however, benchmark-specific bias may still arise from generation templates, schema constraints, or task taxonomy design, and should be considered when interpreting the results. In operational deployments, more granular or domain-specific risk scoring may be required to align temporal measurements with application-level safety policies, alert thresholds, and intervention logic. While coarse, this abstraction improves stability and interpretability of temporal trends. Exploring continuous or task-specific risk representations remains an open direction. Finally, attack trajectories are generated from predefined strategy classes.

Unknown or hybrid attack forms may exhibit different temporal signatures. However, the observed entropy and divergence patterns suggest that many escalation dynamics generalize beyond the specific strategies instantiated here.

A direct evaluation on additional publicly available datasets remains challenging because existing public benchmarks for LLM-agent security are generally designed for prompt injection, indirect prompt injection, or retrieval corruption under step-local settings, and typically do not provide multi-step persistent-memory trajectories with temporal risk annotations. As a result, they are not directly compatible with the trajectory-level temporal measurements introduced in this work without substantial relabeling or synthetic reconstruction. To partially address this external validity concern, we include a qualitative sanity check on a publicly available memory-enabled agent framework, and we view systematic cross-benchmark adaptation to external agent datasets as an important direction for future work.

A. QUALITATIVE SANITY CHECK IN A DEPLOYED AGENT FRAMEWORK

To assess whether the temporal escalation patterns observed in our benchmark are artifacts of controlled generation, we conducted a qualitative sanity check using the *ElizaOS* agent framework [52], which supports persistent memory in conversational agents. This analysis is observational and not intended as a benchmark comparison or quantitative evaluation. We manually replayed a small number of benign, preference-oriented multi-step interaction sequences that incrementally influenced stored memory. Across these trajectories, we observed behavior consistent with our benchmark findings: early interactions appear locally benign, while later responses reflect accumulated memory rather than any single prompt. Behavioral influence emerges in a delayed and non-monotonic manner, reinforcing that temporal memory effects are not confined to synthetic settings. These interactions do not involve malicious payloads, real-world actions, or policy violations. Their purpose is solely to provide qualitative evidence that the temporal dynamics measured in our controlled framework can also manifest in realistic memory-enabled agents. Representative snapshots appear in Appendix X. Although this sanity check is not a substitute for full benchmark transfer, it provides preliminary external validation that the delayed and non-monotonic temporal effects measured in our controlled benchmark can also arise in a publicly available, memory-enabled agent framework beyond the synthetic generation pipeline used in this study.

X. CONCLUSION

Persistent memory fundamentally alters the security landscape of LLM-based agents by allowing adversarial influence to accumulate and activate over extended interaction horizons. Beyond the empirical findings themselves, the paper contributes a trajectory-level evaluation framework and temporal measurement vocabulary for studying persistent-memory compromise in LLM agents. In this work, we presented a large-scale, trajectory-level measurement study of multi-step memory poisoning attacks against stateful LLM agents operating in simulated Web3-style environments. By constructing and analyzing a curated dataset of 2,614 structurally validated attack trajectories, we showed that memory poisoning frequently exhibits delayed and non-monotonic escalation behavior that is poorly captured by step-local or prefix-based evaluation. Our analysis further revealed substantial variation across attack families in how risk unfolds over time, underscoring that temporal structure—not interaction length alone—governs exposure and detectability. Through an early-stop ablation, we demonstrated that truncation-based controls can reduce exposure for some attack classes while introducing systematic blind spots for slow-drift strategies, highlighting the limitations of static step limits. Together, these findings indicate that risk in memory-enabled agents is a temporal property that cannot be reliably assessed using endpoint-only criteria. By releasing our dataset and deterministic measurement pipeline, we aim to support future work on principled evaluation, monitoring, and mitigation of long-horizon risks in memory-enabled agentic systems. An important next step is to adapt the proposed temporal measurement framework to agent-security benchmarks and deployed memory-enabled agent frameworks in order to further assess cross-dataset generalizability.

APPENDIX
QUALITATIVE SANITY CHECK IN A DEPLOYED AGENT ENVIRONMENT

To assess whether the temporal escalation patterns observed in our synthetic benchmarks are artifacts of controlled generation, we conducted a qualitative sanity check using the ElizaOS agent framework, which supports persistent memory in deployed conversational agents. These experiments are qualitative and observational in nature and are not intended as a benchmark comparison or quantitative evaluation.

Figure 15 presents representative snapshots showing preference persistence, delayed recall, and downstream reasoning influence across interaction steps. While early steps appear harmless in isolation, later responses reflect accumulated memory state rather than any single prompt. This example illustrates delayed and non-monotonic behavioral influence consistent with the temporal escalation dynamics measured in our benchmark.

We emphasize that this observation is illustrative only and is not used for performance evaluation, robustness claims, or security assessment. Its purpose is to provide qualitative evidence that the temporal memory effects measured in our controlled setting can also manifest in a realistic memory-enabled agent frameworks.

APPENDIX
REPRODUCIBILITY

Our evaluation relies on three artifacts: (i) a dataset of multi-step memory-poisoning trajectories, (ii) a deterministic

measurement and analysis pipeline, and (iii) scripts that generate every table and figure reported in the paper.

A. DATASET

We used the full set of generated adversarial memory attack traces (2,614 instances) as structured JSONL and CSV files. Each trajectory includes step-level annotations (attack class, injected memory content, activation triggers, and categorical risk labels). All trajectories are validated, deduplicated, and schema-constrained prior to release, ensuring that the dataset is directly usable for reproducing all reported analyses.

B. PIPELINE

The measurement pipeline performs (i) schema validation and normalization, (ii) enforcement of attack-specific structural constraints, (iii) computation of step-level and trajectory-level risk metrics, and (iv) aggregation of publication-ready tables. The pipeline is deterministic, does not rely on external APIs at evaluation time, and uses fixed random seeds where applicable.

C. REPRODUCIBILITY

All figures and tables in the paper are generated directly from the released dataset by running the provided scripts. Each script corresponds one-to-one to a reported figure or table, enabling complete reproduction of experimental results without modifying agent internals or regenerating attack trajectories.

D. ENVIRONMENT

Artifacts run on Python $\geq$ 3.10 using open-source libraries only (e.g., NumPy, Pandas, Matplotlib). No GPU, cloud services, or proprietary software are required.

E. LLM USAGE DISCLOSURE

Some trajectories were produced using an LLM during dataset construction under controlled decoding and schema constraints. Reproducing the reported measurements does not require regenerating trajectories or calling external LLM APIs, as the released dataset contains the finalized attack traces. Generation-time details are documented for transparency and auditing.

APPENDIX
DATASET STATISTICS AND COMPOSITION

This appendix provides descriptive statistics of the final validated dataset used throughout the paper. These statistics are intended to support transparency and reproducibility, and do not introduce additional evaluation results beyond those reported in the main sections (See Figures 16, 17, and 18).

REFERENCES

[1] Z. Zhang, Q. Dai, X. Bo, C. Ma, R. Li, X. Chen, J. Zhu, Z. Dong, and J.-R. Wen, "A survey on the memory mechanism of large language model-based agents," *ACM Trans. Inf. Syst.*, vol. 43, no. 6, pp. 1–47, Nov. 2025.

[2] X. Xu, W. Zhang, Z. Wang, F. Xiao, R. Zheng, Z. Ba, and K. Ren, "RedAgent: An autonomous agent for context-aware red teaming of LLM jailbreaks," *IEEE Trans. Depend. Sec. Comput.*, early access, Feb. 16, 2026, doi: 10.1109/TDSC.2026.3665230.

[3] S. Fan and T. Min, "Web3Agent: Automating on-chain operations via natural language interfaces," *ACM Trans. Web*, vol. 20, no. 1, pp. 1–27, 2025.

[4] J. Sun, A. E. Saddik, and W. Cai, "Smart contract as a service: A paradigm of reusing smart contract in web3 ecosystem," *IEEE Consum. Electron. Mag.*, vol. 14, no. 1, pp. 46–55, Jan. 2025.

[5] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in *Proc. 34th USENIX Secur. Symp. (USENIX Secur.)*, 2024, pp. 2383–2400.

[6] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," 2023, *arXiv:2307.15043*.

[7] Z. Zhang, J. Xu, B. Li, Y. Sun, H. M. Mo, and Y. Chen, "AWMT: Automatic jailbreaking attack framework utilizing working-memory trees," *Expert Syst. Appl.*, vol. 303, Mar. 2026, Art. no. 130643.

[8] Y. Wang, W. Hu, Y. Dong, J. Liu, H. Zhang, and R. Hong, "Align is not enough: Multimodal universal jailbreak attack against multimodal large language models," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 35, no. 6, pp. 5475–5488, Jun. 2025.

[9] D. Shu, C. Zhang, M. Jin, Z. Zhou, and L. Li, "AttackEval: How to evaluate the effectiveness of jailbreak attacking on large language models," *ACM SIGKDD Explorations Newslett.*, vol. 27, no. 1, pp. 10–19, Jul. 2025.

[10] A. Singh Patlan, P. Sheng, S. Ashwin Hebbar, P. Mittal, and P. Viswanath, "Real AI agents with fake memories: Fatal context manipulation attacks on web3 agents," 2025, *arXiv:2503.16248*.

[11] W. Zou, R. Geng, B. Wang, and J. Jia, "PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models," in *Proc. 34th USENIX Secur. Symp. (USENIX Secur.)*, 2024, pp. 3827–3844.

[12] Z. Chen, B. Li, D. Song, Z. Xiang, and C. Xiao, "AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 130185–130213.

[13] R. Zhang, H. Li, R. Wen, W. Jiang, Y. Zhang, M. Backes, Y. Shen, and Y. Zhang, "Instruction backdoor attacks against customized LLMs," in *Proc. 33rd USENIX Secur. Symp. (USENIX Secur.)*, 2024, pp. 1849–1866.

[14] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomelí, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 68539–68551.

[15] T. Yu, G. Liu, C. Wang, and Y. Yang, "LLMGraph: Label-free detection against APTs in edge networks via LLM and GCN," *IEEE Trans. Depend. Sec. Comput.*, vol. 22, no. 6, pp. 7256–7271, Nov./Dec. 2025.

[16] Y. Gong, D. Ran, X. He, T. Cong, A. Wang, and X. Wang, "Safety misalignment against large language models," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2025.

[17] Z. Xi et al., "The rise and potential of large language model based agents: A survey," *Sci. China Inf. Sci.*, vol. 68, no. 2, 2025, Art. no. 121101.

[18] F. Cassano, A. Gopinath, K. Narasimhan, N. Shinn, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 8634–8652.

[19] M. Kravchik, L. Demetrio, B. Biggio, and A. Shabtai, "Practical evaluation of poisoning attacks on online anomaly detectors in industrial control systems," *Comput. Secur.*, vol. 122, Nov. 2022, Art. no. 102901.

[20] M. Goldblum, D. Tsipras, C. Xie, X. Chen, A. Schwarzschild, D. Song, A. Madry, B. Li, and T. Goldstein, "Dataset security for machine learning: Data poisoning, backdoor attacks, and defenses," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 2, pp. 1563–1580, Feb. 2023.

[21] G. Li, C.-C. Lin, X. Zhang, X. Ma, and L. Guo, "Adversarial robust ViT-based automatic modulation recognition in practical deep learning-based wireless systems," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2025, pp. 3672–3690.

[22] H. Yang, K. Xiang, M. Ge, H. Li, R. Lu, and S. Yu, "A comprehensive overview of backdoor attacks in large language models within communication networks," *IEEE Netw.*, vol. 38, no. 6, pp. 211–218, Nov. 2024.

[23] Y. Zhang, T. Du, Y. Ma, X. Wang, Y. Xie, G. Yang, Y. Lu, and E. Chang, "AttacKG+: Boosting attack graph construction with large language models," *Comput. Secur.*, vol. 150, Mar. 2025, Art. no. 104220.

[24] M. Bosma, E. Chi, B. Ichter, Q. V. Le, D. Schuurmans, X. Wang, J. Wei, F. Xia, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," *Adv. Neural Inf. Process. Syst.*, pp. 24824–24837, Nov./Dec. 2022.

[25] N. Carlini, F. Tramèr, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. P. Roberts, T. B. Brown, D. Song, Ú. Erlingsson, A. Oprea, and C. Raffel, "Extracting training data from large language models," in *Proc. 30th USENIX Secur. Symp. (USENIX Secur.)*, 2020, pp. 2633–2650.

[26] Q. Zhang, B. Zeng, C. Zhou, G. Go, H. Shi, and Y. Jiang, "Human-imperceptible retrieval poisoning attacks in LLM-powered applications," in *Companion Proc. 32nd ACM Int. Conf. Found. Softw. Eng.*, 2024, pp. 502–506.

[27] M. Herrador and J. Rehberger, "SpAIware: Uncovering a novel artificial intelligence attack vector through persistent memory in LLM applications and agents," *Future Gener. Comput. Syst.*, vol. 174, Jan. 2026, Art. no. 107994.

[28] H. Zhang, Y. Gao, W. Hu, G. Li, J. Pang, and J. Li, "ReSLC: Defending backdoor attacks on intelligent vulnerability detection via redundant semantic LLM compression," *J. Inf. Secur. Appl.*, vol. 100, Jul. 2026, Art. no. 104430. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2214212626000608

[29] E. Hubinger et al., "Sleeper agents: Training deceptive LLMs that persist through safety training," 2024, *arXiv:2401.05566*.

[30] E. Debenedetti, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2024.

[31] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Proc. Findings Assoc. Comput. Linguistics ACL*, 2024, pp. 10471–10506.

[32] A. S. Patlan, P. Viswanath, and P. Mittal, "Locks tested without burglars: Using coding assistants to break prompt injection defenses," in *Proc. NeurIPS Workshop, Reliable ML Unreliable Data*, 2025.

[33] M. Leo, F. Tan, T. Miao, and G. Anand, "From threat to trust: Assessing security risks of agentic AI systems," *Int. J. Inf. Secur.*, vol. 25, no. 1, p. 23, Feb. 2026, doi: 10.1007/s10207-025-01185-y.

[34] M. Andreoni, W. T. Lunardi, G. Lawton, and S. Thakkar, "Enhancing autonomous system security and resilience with generative AI: A comprehensive survey," *IEEE Access*, vol. 12, pp. 109470–109493, 2024.

[35] A. Chhabra, S. Datta, S. K. Nahin, and P. Mohapatra, "Agentic AI security: Threats, defenses, evaluation, and open challenges," *IEEE Access*, vol. 14, pp. 49455–49482, 2026.

[36] F. He, T. Zhu, D. Ye, B. Liu, W. Zhou, and P. S. Yu, "The emerged security and privacy of LLM agent: A survey with case studies," *ACM Comput. Surveys*, vol. 58, no. 6, pp. 1–36, Apr. 2026.

[37] P. Cheng, Z. Wu, W. Du, H. Zhao, W. Lu, and G. Liu, "Backdoor attacks and countermeasures in natural language processing models: A comprehensive security review," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 8, pp. 13628–13648, Aug. 2025.

[38] A. Yazdinejad, H. Karimipour, and T. Halabi, "Toward stress-adaptive cyber defense: Cognitive–physiological synchronization in IoT environments," *IEEE Internet Things J.*, vol. 13, no. 7, pp. 13832–13848, Apr. 2026.

[39] S. Kumari and G. Yadav, "Vuln2Action: An LLM-based framework for generating vulnerability reproduction steps and mapping exploits," *J. Inf. Secur. Appl.*, vol. 99, Jun. 2026, Art. no. 104420.

[40] M. Yu, F. Meng, X. Zhou, S. Wang, J. Mao, L. Pang, T. Chen, K. Wang, X. Li, Y. Zhang, B. An, and Q. Wen, "A survey on trustworthy LLM agents: Threats and countermeasures," in *Proc. 31st ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2025, pp. 6216–6226.

[41] A. Liu, Y. Zhou, X. Liu, T. Zhang, S. Liang, J. Wang, Y. Pu, T. Li, J. Zhang, W. Zhou, Q. Guo, and D. Tao, "Compromising LLM driven embodied agents with contextual backdoor attacks," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 3979–3994, 2025.

[42] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo, "SecAlign: Defending against prompt injection with preference optimization," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2025, pp. 2833–2847.

[43] I. Evtimov, A. Zharmagambetov, A. Grattafiori, C. Guo, and K. Chaudhuri, "WASP: Benchmarking Web agent security against prompt injection attacks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 38, 2026.

[44] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *Proc. 11th Int. Conf. Learn. Represent.*, 2022.

[45] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. 36th Annu. ACM Symp. Interface Softw. Technol.*, 2023, pp. 1–22.

[46] H. Chaudhari, J. Hayes, M. Jagielski, I. Shumailov, M. Nasr, and A. Oprea, "Cascading adversarial bias from injection to distillation in language models," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2025, pp. 4409–4422.

[47] A. Musamih, I. Yaqoob, K. Salah, R. Jayaraman, and M. Omar, "Blockchain for large language models (LLMs): Applications, challenges, and framework implementation," *Expert Syst. Appl.*, vol. 308, May 2026, Art. no. 131100.

[48] OpenAI. (2024). *GPT-4o Mini*. [Online]. Available: https://platform.openai.com/docs/models

[49] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Secur. Privacy*, May 2010, pp. 305–316.

[50] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, and E. Wong, "Jailbreaking black box large language models in twenty queries," in *Proc. IEEE Conf. Secure Trustworthy Mach. Learn. (SaTML)*, Apr. 2025, pp. 23–42.

[51] X. Wang, D. Wu, Z. Ji, Z. Li, P. Ma, S. Wang, Y. Li, Y. Liu, N. Liu, and J. Rahmel, "SelfDefend: LLMs can defend themselves against jailbreaking in a practical manner," in *Proc. 34th USENIX Secur. Symp. (USENIX Secur.)*, 2025, pp. 2441–2460.

[52] S. Walters, S. Gao, S. Nerd, F. Da, W. Williams, T.-C. Meng, A. Chow, H. Han, F. He, A. Zhang, M. Wu, T. Shen, M. Hu, and J. Yan, "Eliza: A web3 friendly AI agent operating system," 2025, *arXiv:2501.06781*.



ABBAS YAZDINEJAD is currently an Assistant Professor with the Department of Computer Science, University of Regina, and the Director of the Decentralized Cybersecurity & Artificial Intelligence Laboratory (DCAILab). He is also an Adjunct Professor with the Department of Electrical and Software Engineering, University of Calgary, AB, Canada, and a Balsillie Scholar with the Balsillie School of International Affairs (BSIA), Waterloo, Canada. He has been recognized among the World's Top 2% Scientists (Stanford University ranking). His research interests include artificial intelligence and machine learning for critical and safety- and security-sensitive applications, with a particular emphasis on agentic AI systems, large language model (LLM) security and governance, autonomous cybersecurity, privacy-preserving machine learning, federated learning, the Internet of Things (IoT) and Industrial IoT, and quantum computing. He has held postdoctoral research positions with the University of Toronto and the University of Guelph. His work bridges technical innovation, teaching, and industry-engaged research to advance trustworthy and resilient digital systems.



HADIS KARIMIPOUR received the Ph.D. degree in electrical engineering from the University of Alberta, in 2016. She is currently a Canada Research Chair (Tier II) and an Associate Professor with the Department of Electrical and Software Engineering, University of Calgary, where she also works as the Director of the Smart Cyber-Physical Systems (SCPS) Laboratory. Her research interests include AI-enabled cybersecurity for critical infrastructures, the IoT/OT systems, and the development of resilient cyber-physical grids. She is internationally recognized for her pioneering work at the intersection of artificial intelligence and cybersecurity, with over 135 peer-reviewed publications and more than 7000 citations. Her research has attracted nearly nine million CAD in funding from governmental and industrial agencies. She has received numerous prestigious honors, including the WXN Canada's Most Powerful Women: the IEEE Early Achievement Award, in 2022, Top 100 Award, in 2024, and the Top 25 Women of Influence Award, in 2025. She actively contributes to national and international cybersecurity policy, serves on several expert panels, and is an Associate Editor for leading journals, including IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS and the *International Journal of Electrical Power & Energy Systems*.